



**Politecnico
di Torino**

Microelectronic Systems

Group 02

DLX Microprocessor
Design & Development
Final Project Report

Master degree in Electronics Engineering

Master degree in Computer Engineering

Referents: Prof. Mariagrazia Graziano, Prof. Giovanna Turvani

Authors:

Sabina Sarcuni, Leonardo Gallina, Francesco Mignone

July 19, 2025

Abstract

This document provides a comprehensive overview of the design and development of a DLX microprocessor, a project undertaken by Group 02 as part of the Microelectronic Systems course at Politecnico di Torino. The project aims to enhance understanding of microprocessor design stack: starting from a RTL VHDL description to the Physical Design process. The report details the structure and the process that lead to the implementation. To allow the reader to understand the design choices, the report is structured as follows:

- **Chapter 1: Introduction** - Introduces the DLX microprocessor architecture, its instruction set, and the project objectives.
- **Chapter 2: DLX Structure** - Discusses the design process, including the control unit, Datapath, and functional units.
- **Chapter 3: Synthesis** - Describes the testing methodologies used to validate the design and ensure correctness.
- **Chapter 4: Physical Design** - Covers the synthesis process and the physical design steps taken to prepare the microprocessor for fabrication.
- **Chapter 5: Tools and Scripts** - Lists the tools and technologies used throughout the project, including VHDL, ModelSim/Questasim, Innovus, Docker and custom scripts.
- **Chapter 6: Conclusions** - Summarizes the project outcomes, challenges faced, and lessons learned.
- **Appendices** - Provides additional resources, including VHDL code, user scripts, and TCL templates used in the project.

Prior knowledge of VHDL and digital design principles is assumed, as the report focuses on the implementation details and design choices made during the project. Also a general knowledge of computer architectures is recommended, mostly in RISC principles pipelined architectures. The document serves as a valuable resource for understanding the complexities of microprocessor design and the practical application of theoretical concepts in a real-world project.

Contents

Abstract	i
1 Introduction	1
1.1 Project Objectives	1
1.2 Tools used	1
1.3 The DLX Microprocessor	2
1.3.1 DLX Architecture	2
1.3.2 Instruction Set	2
1.3.3 DLX Base Version	3
1.4 DLX Pro Features	4
2 DLX Project Structure	5
2.1 Control Unit	5
2.1.1 Control Word	5
2.2 Datapath	7
2.2.1 Fetch Stage	8
2.2.2 Decode Stage	9
2.2.3 Execute Stage	11
2.2.4 Memory and Write Back Stage	11
2.2.5 Forwarding Unit	12
2.3 Simulation, Synthesis and Physical Design	13
3 Synthesys	14
3.1 Synthesis Process	14
3.2 Results	15
4 Physical Design	17
4.1 Results	18
4.1.1 Area and Gate Count	18
4.1.2 Timing Analysis	18
4.1.3 Conclusion	19
5 Tools and Scripts	20
5.1 Docker Container	20
5.2 Marley	20
5.2.1 Pupi	21
5.2.2 Briciola	21
5.2.3 Scorzetta	21
5.2.4 Boh	21

5.2.5	Cassiopeia	21
6	Conclusions	22
6.1	Further Work	22
A	DLX VHDL Code	23
A.1	Control unit	23
A.2	DataPath	30
A.3	Fetch Stage	40
A.4	Decode Stage	42
A.5	Execute Stage	46
A.6	Memory and Write Back Stage	50
A.7	Forwarding Unit	52
B	User Scripts	54
B.1	Pupi	54
B.2	Marley	57
B.3	Briciola	60
B.4	Scorzetta	61
B.5	Boh	65
B.6	Cassiopeia	69
C	TCL templates	71
C.1	Simualtion	71
C.2	Synthesys	72
C.3	Physical Design	74
C.3.1	Default.view	74
C.3.2	DLX.globals	74
C.3.3	Physical Design	75

CHAPTER 1

Introduction

1.1 Project Objectives

The primary objectives of the DLX project are as follows:

- To design and implement a functional DLX microprocessor using VHDL.
- To understand the principles of RISC architecture and pipeline processing.
- To validate the design through simulation.
- Synthesize the processor.
- Use a standard cell layout for Physical Design.
- Document the design process, challenges faced, and lessons learned.

1.2 Tools used

The project utilizes several tools and technologies to facilitate the design, simulation, and testing of the DLX microprocessor:

- **VHDL (VHSIC Hardware Description Language):** The primary language used for describing and designing the microprocessor components like the control unit, Datapath, and functional units.
- **ModelSim/QuartaSim:** A simulation tool used to test and validate the VHDL code.
- **Design Compiler:** A synthesis tool used to convert the VHDL code into a gate-level netlist, optimizing the design on given constraints.
- **Innovus:** A tool used for physical design starting from a synthesized netlist and constraints.
- **Docker:** A containerization platform used to create a consistent development environment, ensuring that all team members can work with the same tools and configurations.
- **GitHub:** A version control platform used for collaboration and code management throughout the project.
- **Marley & Co.:** A set of tools developed by the team to facilitate and automatize the design and testing of the DLX microprocessor:

- **Marley**: Main script that helps set up the user environment and manage the other scripts.
- **Pupi**: program that generates the compilation list based on the dependencies between the VHDL files.
- **Briciola**: script that compiles the VHDL files with Modelsim or QuestaSim, generating the necessary libraries and simulation files.
- **Scorzetta**: script that runs the simulation of a given testbench.
- **Boh**: script that runs the synthesis of an entity, generating the synthesized netlist and constraints.
- **Cassiopeia**: script that runs the physical design of the synthesized netlist, generating the final layout and design files.

1.3 The DLX Microprocessor

The DLX microprocessor is a simplified RISC (Reduced Instruction Set Computer) Harvard architecture designed to facilitate the understanding of microprocessor design principles. It serves as an educational tool, allowing to explore the fundamental concepts of computer architecture, instruction execution, and VHDL coding practices. The DLX architecture is characterized by its five-stage pipeline, which includes: Fetch, Decode, Execute, Memory Access, and Write Back stages. This design enables efficient instruction processing and enhances overall throughput.

1.3.1 DLX Architecture

As DLX architecture is based on a simplified RISC design, which emphasizes a small set of instructions and a streamlined execution process. The DLX microprocessor employs a five-stage pipeline architecture, which allows for In-Order instruction execution. The stages are:

- **Fetch (IF)**: Retrieve the instruction from the Instruction Memory given the current Program Counter (PC).
- **Decode (ID)**: Decode the instruction to fetch the necessary operands.
- **Execute (EX)**: Perform the operation specified by the instruction.
- **Memory Access (MEM)**: Access the Data Memory if required by the instruction.
- **Write Back (WB)**: Write the result back to the register file if needed.

Each stage of the pipeline, and its implementation, will be discussed in detail in the following sections. The DLX architecture also includes a register file with 32 general-purpose registers, each 32 bits wide, and, being based on a Harvard Architecture, a dedicated instruction memory (IRAM) and data memory (DRAM) for storing instructions and data, respectively.

1.3.2 Instruction Set

The DLX instruction set is designed to be simple and efficient, adhering to the principles of RISC architecture. It includes instructions for arithmetic operations, logical operations, data movement, and control flow. The instruction set is divided into several categories:

- **Arithmetic Instructions**: These instructions perform basic arithmetic operations such as addition, subtraction, multiplication, and division.

- **Logical Instructions:** These instructions perform bitwise operations such as AND, OR, NOT, and XOR.
- **Data Movement Instructions:** These instructions are used to move data between registers and memory, including load and store operations.
- **Control Flow Instructions:** These instructions alter the flow of execution, including branches and jumps.

The RISC nature of the DLX instruction set allows for a reduced number of instructions, which simplifies the design and implementation of the microprocessor. Each instruction is encoded in a fixed-length format, typically 32 bits, which facilitates efficient decoding and execution. They are divided into three main formats:

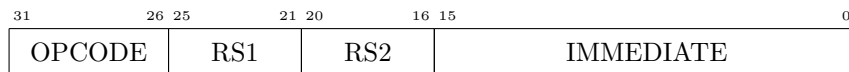


Figure 1.1: I-Type Instruction Format

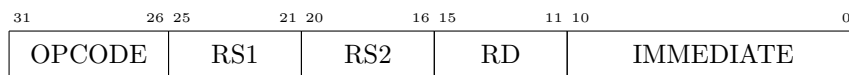


Figure 1.2: R-Type Instruction Format

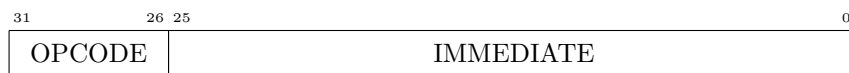


Figure 1.3: J-Type Instruction Format

- **R-type (Register):** These instructions operate on registers and include operations like addition, subtraction, and logical operations. They typically have three register operands.
- **I-type (Immediate):** These instructions include an immediate value as one of the operands, allowing for operations that involve constants. They are used for operations like loading immediate values into registers or performing arithmetic with an immediate value.
- **J-type (Jump):** These instructions are used for control flow, allowing the program to jump to a specific address in memory. They are essential for implementing loops and conditional statements.
- **Branch Instructions:** These instructions alter the flow of execution based on conditions, such as equality or inequality checks. They are used to implement loops and conditional statements.
- **Load/Store Instructions:** These instructions are used to transfer data between registers and memory. Load instructions read data from memory into a register, while store instructions write data from a register to memory.

1.3.3 DLX Base Version

The base version of the DLX microprocessor is designed to implement the core functionality of the DLX architecture. It includes the essential components required for instruction execution, such as the control unit, Datapath, and functional units. The base version serves as a foundation for further enhancements and optimizations, allowing for a clear understanding of the microprocessor's operation.

Base Instruction Set

The base version of the DLX microprocessor supports a limited set of instructions, focusing on the fundamental operations required for basic computation. The instruction set includes:

- **Arithmetic Instructions: R-type:** ADD, SUB, SLL, SRL, SNE, SLE, SGE
- **Arithmetic Instructions: I-type:** ADDi, SUBi, SLLi, SRLi, SNEi, SLEi, SGEi
- **Logical Instructions: R-Type:** AND, OR, NOT, XOR
- **Logical Instructions: I-Type:** ANDi, ORi, NOTi, XORi
- **Memory Instructions: R-Type:** LOAD (LW), STORE (SW)
- **Control Flow Instructions:** JUMP, JAL, BNEZ, BEQZ

These instructions are sufficient to perform basic arithmetic and logical operations, as well as control flow operations, allowing for the execution of simple programs.

1.4 DLX Pro Features

We implemented some features starting from the base version of DLX processor to enhance its functionality and performance. These features include:

- **Forwarding Unit:** This unit is designed to handle data hazards by forwarding the results of previous instructions to subsequent instructions that require them, thus reducing the need for stalls in the pipeline.
- **Branch Prediction:** The DLX microprocessor incorporates a simple static branch prediction mechanism to improve the performance of control flow instructions. This feature helps to minimize pipeline stalls caused by branch instructions.

CHAPTER 2

DLX Project Structure

The project started with a briefing on the DLX architecture to set how we wanted to structure the project. We decided to split the architecture in smaller parts, to ease the workload among the group. This also brings a more modular design, which is easier to test and debug and improve.

The first step was to split the architecture into two main components: the Control Unit (CU) and the Datapath (DP).

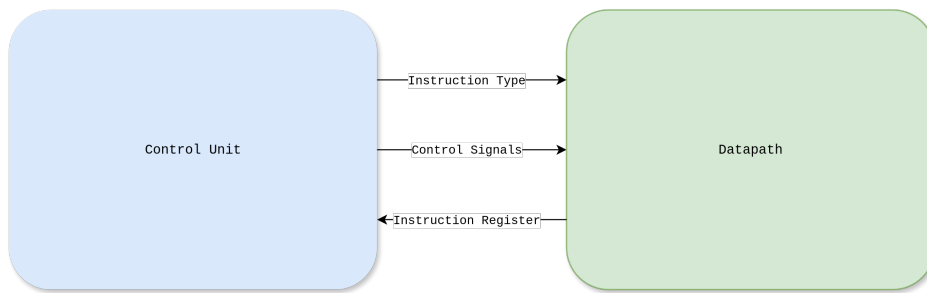


Figure 2.1: DLX Internal Structure

2.1 Control Unit

The Control Unit (CU) 2.2 is responsible for generating the control signals that orchestrate the operation of the Datapath. It interprets the instruction opcode and generates the necessary signals to control the execution of the instruction. The chosen type of CU is a Hardwired Control Unit, which uses combinational logic to generate control signals (Control Word - CW) at each new instruction fetch. At each cycle the OPCODE is decoded and used to address a ROM that contains the control signals for all the Datapath phases. The Control Word is then skewed through a series of registers to generate the control signals for each subsequent stage of the Datapath for at least one clock cycle.

2.1.1 Control Word

The control word is a 15-bit vector that contains the needed control signals for the Datapath. As stated before it is generated based on the instruction opcode. Each bit in the control word corresponds to a specific control signal:

- **F_e** : Fetch Enable, used to enable the instruction fetch stage.

- **D_e** : Decode Enable, used to enable the decode stage.
- **RF_e** : Register File Enable, used to enable register file operations.
- **RF_re_1** : Register File Read Enable 1, used to enable reading from the first source register.
- **RF_re_2** : Register File Read Enable 2, used to enable reading from the second source register.
- **Ex_e** : Execute Enable, used to enable the execution stage.
- **S_1_s** : Source 1 Select, used to select the first source for the ALU operation.
- **S_2_s** : Source 2 Select, used to select the second source for the ALU operation.
- **M_e** : Memory Enable, used to enable memory stage.
- **DRAM_e** : DRAM Enable, used to enable the DRAM.
- **DRAM_re** : DRAM Read Enable, used to enable reading from the DRAM.
- **DRAM_we** : DRAM Write Enable, used to enable writing to the DRAM.
- **S_3_s** : Source 3 Select, used to select the source to forward to the Write Back stage.
- **RF_we** : Register File Write Enable, used to enable writing to the register file and to force an enable on the register file.

And the Codeword memory is a ROM filled with the following values, where each row corresponds to a specific instruction and the columns correspond to the control signals described above. The first column is the instruction type, the second column is the OPCODE, and the remaining columns are the control signals

Instruction	MSB		Control Word												LSB	
	OPCODE	F_e	D_e	RF_e	RF_re_1	RF_re_2	Ex_e	S_1_s	S_2_s	M_e	DRAM_e	DRAM_re	DRAM_we	S_3_s	RF_we	RF_en
R-Type	0x00	1	1	1	1	1	1	0	0	1	0	0	0	0	1	1
addi	0x01	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
subi	0x0A	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
ori	0x0D	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
andi	0x0C	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
xori	0x0E	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
slli	0x14	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
srli	0x16	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
snei	0x19	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
slei	0x1C	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
sgei	0x1D	1	1	1	1	0	1	0	1	1	0	0	0	0	1	1
sw	0x2B	1	1	1	1	0	1	0	1	1	1	0	1	0	0	0
lw	0x23	1	1	1	1	0	1	0	1	1	1	1	0	1	1	1
jmp	0x02	1	1	0	0	0	1	1	1	0	0	0	0	0	0	0
jal	0x03	1	1	0	0	0	1	1	1	1	0	0	0	0	1	1
beqz	0x04	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0
bnez	0x05	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0

Table 2.1: Control Word Memory

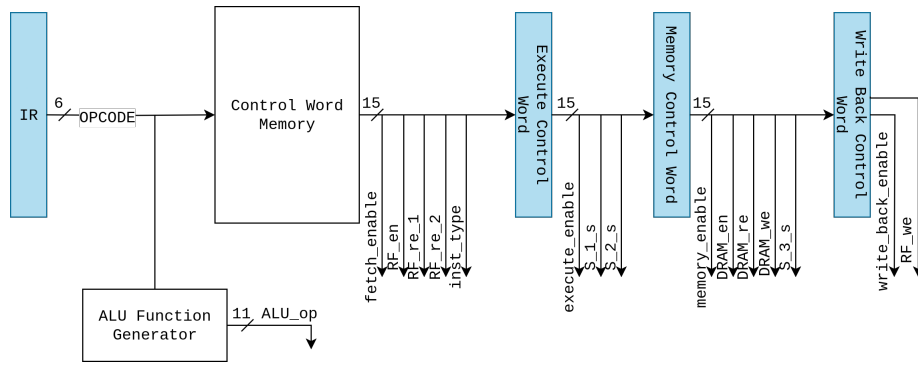


Figure 2.2: DLX Control unit Structure

The ALU operations are assigned based on the type of the instruction:

- **R-Type Instructions:** The ALU performs the operation specified by the function code taken from the current Instruction Register.
- **All other instructions:** based on the OPCODE the correct operation is assigned. Ex. for the **addi** instruction the ALU performs an addition to add the immediate value, for the **sw** instruction it performs an addition to calculate the address to write to.

2.2 Datapath

The Datapath (DP) is the core of the DLX microprocessor, responsible for executing instructions and performing data operations. As shown in Figure 2.3, the Datapath consists of several functional units that work together to execute instructions. Let's recall the main phases of the pipeline:

- **Fetch Stage:** The instruction is fetched from the instruction memory (IRAM) based on the current program counter (PC).
- **Decode Stage:** The fetched instruction is decoded to determine the operation to be performed and the operands involved.
- **Execute Stage:** The ALU performs the specified operation on the operands.
- **Memory Stage:** If the instruction involves memory access, data is read from or written to the data memory (DRAM).
- **Write Back Stage:** The result of the operation is written back to the register file.

The instructions are performed on 32-bit data, and the immediate values are also sign extended to 32 bits. The basic execution based on the type of instruction is as follows:

- **R-Type Arithmetic/Logical Instructions:**

$$RS1 \text{ op } RS2 \rightarrow RD$$

- **I-Type Arithmetic/Logical Instructions:**

$$RS1 \text{ op } Immediate \rightarrow RD$$

- **R-Type Memory Instructions:**

$$RS1 \rightarrow MEM[RS2 + Immediate]$$

- **J-Type Instructions:**

$$\text{PC} + \text{Immediate} \rightarrow PC$$

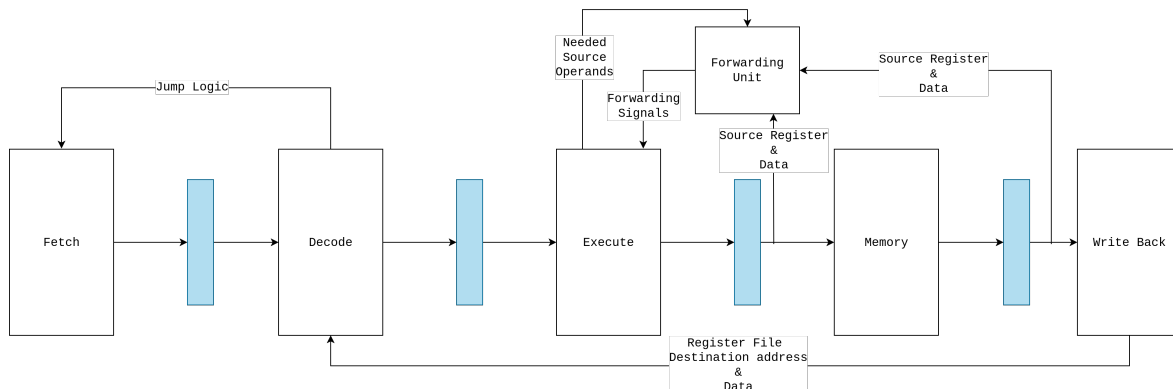


Figure 2.3: DLX Datapath Structure

We structured Datapath by developing each stage separately, allowing us to test each stage independently before integrating them into the full pipeline. This also allows to easily add new features or modify existing ones without affecting the entire design. The stages are connected through a series of registers that hold the intermediate results, ensuring that data flows smoothly through the pipeline.

2.2.1 Fetch Stage

In the Fetch Stage 2.4, the instruction is fetched from the instruction memory (IRAM) based on the current program counter (PC). The PC is incremented by 1 after each fetch to point to the next instruction, it is only incremented by one and not 4 as the IRAM data output is on 32 bits so each memory location is a new instruction. The fetched instruction is stored in the Instruction Register (IR) for further processing in the decode stage and the Control Unit. The fetch stage also includes a branch target address calculation for branch instruction: if the instruction is a branch instruction, the target address is calculated based on the immediate value forwarded from the decode stage and the current PC. The target address is then used to update the PC in the next cycle if the branch is taken (jmp_en signal active high).

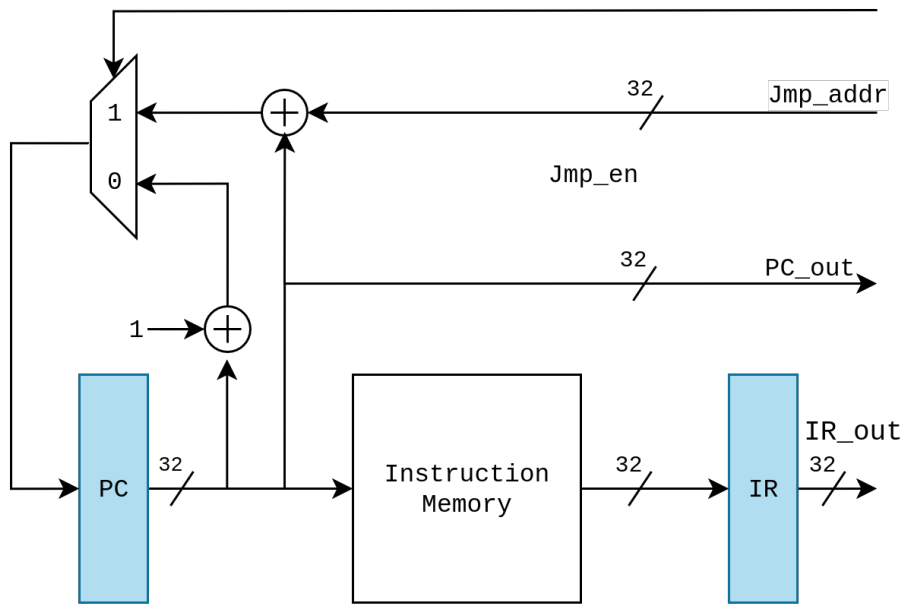


Figure 2.4: Fetch Unit Structure

2.2.2 Decode Stage

During the Decode Stage 2.5, the fetched instruction is decoded to determine the operation to be performed and the operands involved. The instruction is split into its components: opcode, source registers addresses (RS1 and RS2) for the register file, destination register (RD), and immediate value. The decode stage also includes reading the source registers from the register file, which provides the operands for the ALU operation in the execute stage.

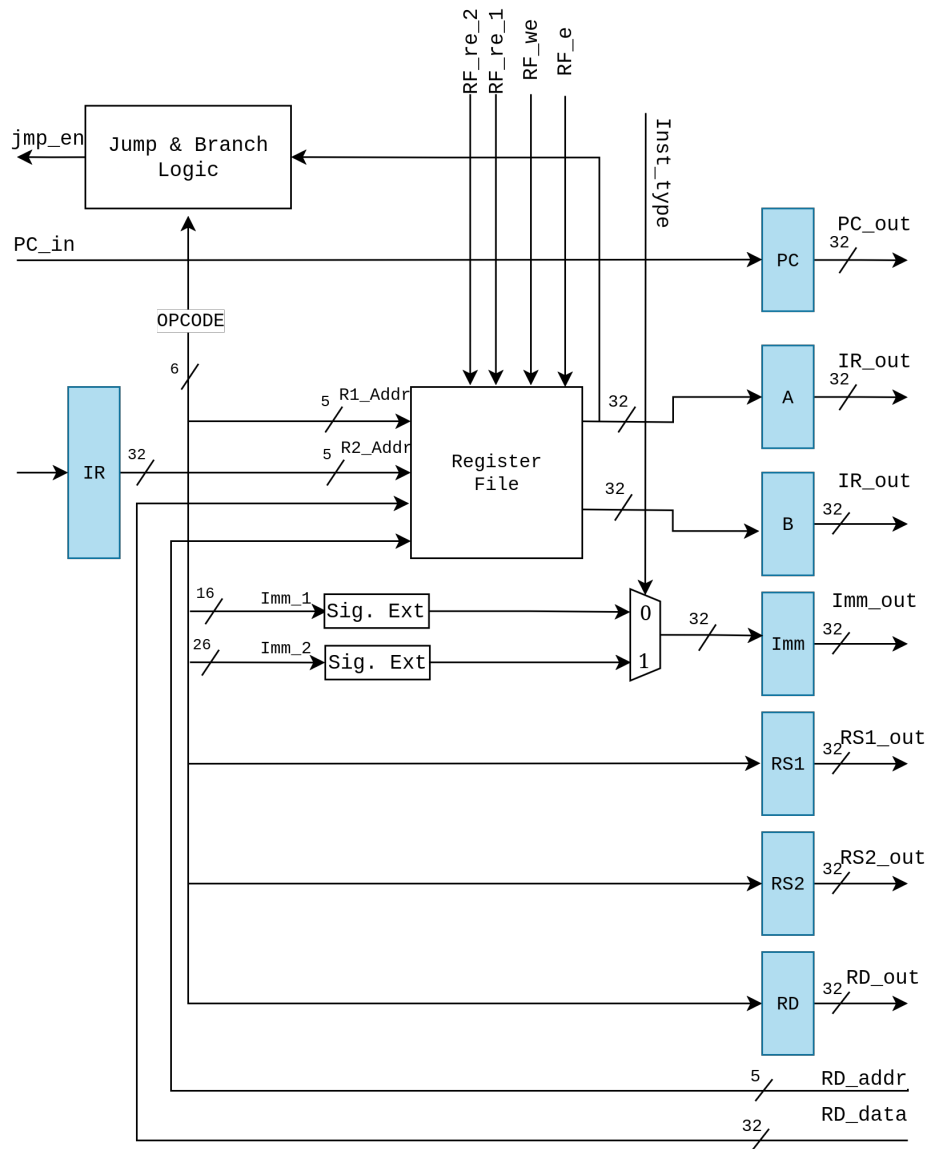


Figure 2.5: Decode Unit Structure

Furthermore a static branch prediction mechanism is added, which once decoded the **beqz** and **bnez** instructions it predicts the branch outcome. The prediction is based on the value of the source register (RS1). If RS1 is zero, the branch is predicted to be taken, otherwise it is predicted not to be taken. This prediction is used to update the PC in the fetch stage, and forcing the next fetched instruction to be a NOP to flush only one stage of the pipeline. In case of a JMP or JAL instruction, the PC is updated always as there are no condition to check. The signal that states if the PC needs to be updated with the jump address is called **jmp_en**. The immediate value is also sign-extended to 32 bits depending on the type of instruction. For R-Type instructions, the immediate value is not used. Meanwhile for I-Type instructions, the first 16th bits are taken, for J-Type instructions, the first 26th bits are taken and sign extended. The source registers are read from a register file. The register file has two read ports and one write port, allowing simultaneous reading and writing of registers. The read ports are used to read the source registers (RS1 and RS2) for the ALU operation, while the write port is used to write the result back to the register file in the Write-Back stage.

2.2.3 Execute Stage

The execute stage 2.6 is where the actual computation takes place. The ALU performs the operation specified by the **ALU_op** signal, and using the operands read from the decode stage or the one forwarded by the forwarding unit. The ALU can perform various operations, including addition, subtraction, logical operations, and shifts. The result of the ALU operation is then forwarded to the next stage of the pipeline, which may involve writing it back to a register or accessing memory.

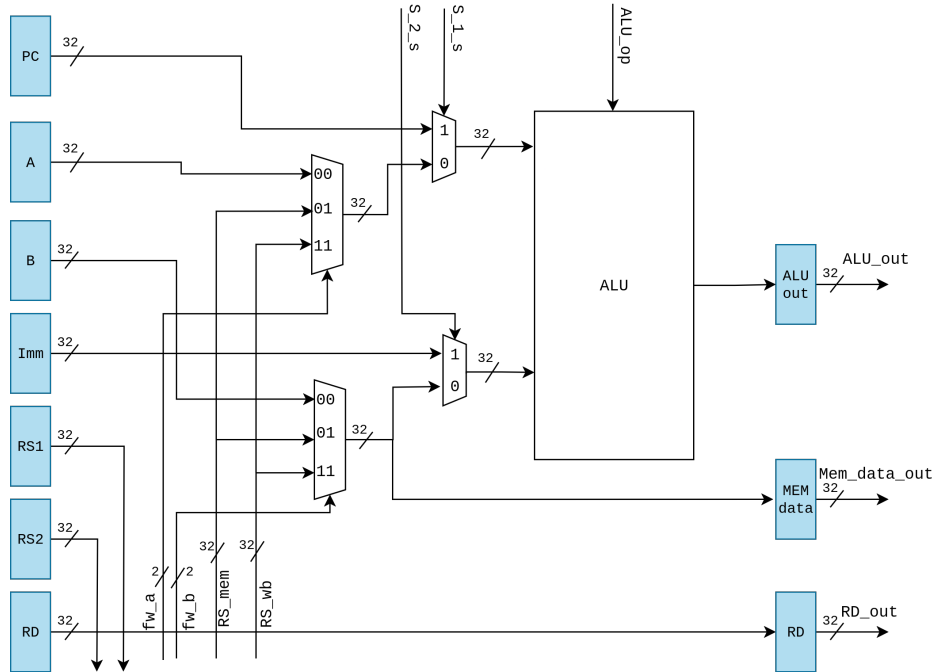


Figure 2.6: Execute Unit Structure

2.2.4 Memory and Write Back Stage

The last stage of the pipeline is the Memory and Write Back Stage 2.7. In this stage, the result of the ALU operation is written back to the register file if the instruction requires it. If the instruction involves memory access (load or store), the data is read from or written to the data memory (DRAM). The DRAM is accessed based on the address calculated in the execute stage, which involve adding an immediate value to a base address stored in a register.

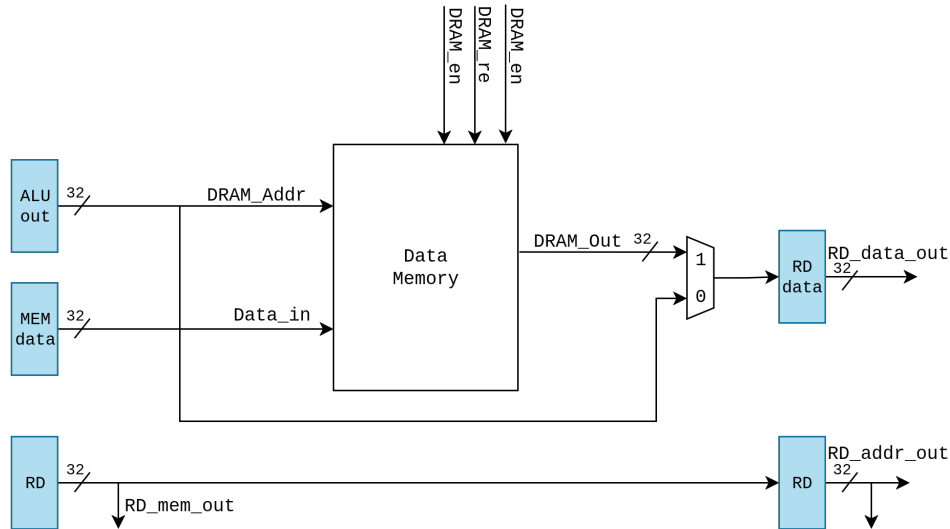


Figure 2.7: Memory and Write Back Unit Structure

2.2.5 Forwarding Unit

The Forwarding Unit 2.8 is responsible for resolving data hazards in the pipeline. It detects when an instruction in the execute stage needs a value that is not yet available in the register file because it is being written back by a previous instruction. In such cases, the forwarding unit forwards the result of the ALU operation directly to the execute stage, bypassing the register file or the memory phase. This allows the pipeline to continue executing instructions without stalling, improving performance and throughput.

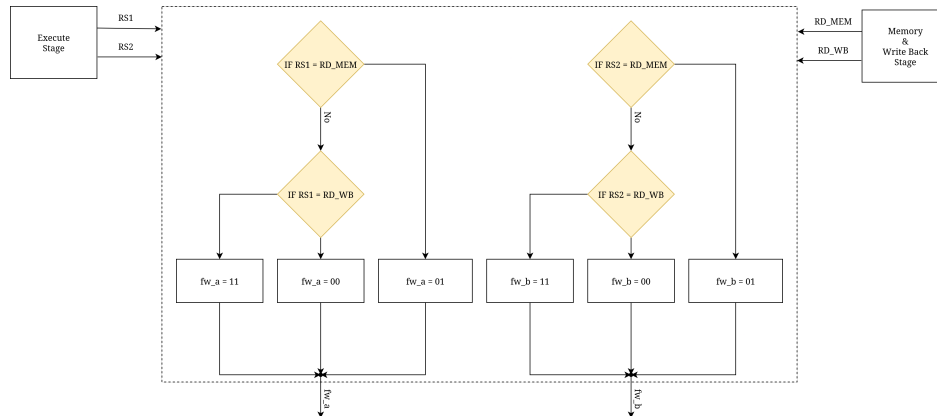


Figure 2.8: Forwarding Unit Flow Chart

To detect the hazards, the forwarding unit checks the destination register (RD) of the instruction in the memory and write back stage against the source registers (RS1 and RS2) of the instruction in the execute stage. If there is a match, it asserts the appropriate forwarding signals to select the correct source for the ALU operation: **fw_a** for source 1 and for source 2 **fw_b**. The forwarding unit also handles cases where the instruction in the memory stage is a load instruction, which requires the data to be forwarded to the execute stage, only in this case we assert a **stall** signal to stall the pipeline for one clock cycle to simulate the DRAM access.

2.3 Simulation, Synthesis and Physical Design

The simulation, synthesis, and physical design of the DLX microprocessor were performed using a series of custom scripts and templates. These scripts automate the process of compiling, simulating, synthesizing, and physically designing the microprocessor, allowing for a more efficient development workflow. To run the desired scripts just run the command

```
./marley.sh
```

and follow the instructions on screen. The scripts are designed to be modular and customizable, allowing users to easily adapt them to their specific needs. For more information on the scripts and their usage, refer to the dedicated chapter 5.

CHAPTER 3

Synthesys

In this chapter we will discuss the synthesis process of the DLX microprocessor. The synthesis is a crucial step in the design flow, where the high-level RTL description is transformed into a gate-level netlist that can be used for further analysis and physical design. The synthesis process involves several key steps, including the analysis of the RTL code, optimization of the design for performance and area, and the generation of a netlist that can be used for physical design. The synthesis tool used in this project is Synopsys Design Compiler. The synthesis process is guided by a set of constraints that define the target technology, timing requirements, and area constraints. These constraints are specified in a Synopsys Design Constraints (SDC) file or set by commands in a TCL file, which is used by the synthesis tool to optimize the design accordingly. The synthesis process also includes the generation of reports that provide insights into the design's performance, area, and power consumption. These reports are essential for evaluating the design and making necessary adjustments to meet the project requirements.

3.1 Synthesis Process

The synthesis process of the DLX microprocessor has been made with a top down approach. Starting from an unconstrained synthesis, we progressively applied more constraints to the design to optimize it for specific performance metrics:

- **Synthesis with no constraints:** This is the baseline synthesis, where the design is synthesized without any specific optimizations applied. It serves as a reference point for comparing the results of other synthesis runs.
- **Synthesis with clock constraint:** In this case, the synthesis tool is instructed to optimize the design for a specific clock period. This means that the design will be optimized to meet the timing requirements for the specified clock period, ensuring that all paths in the design can operate correctly at the target frequency.
- **Synthesis with loose clock and path constraint:** This synthesis run applies both a clock constraint and a path constraint. The clock constraint ensures that the design meets the timing requirements for the specified clock period, while the path constraint allows for more fine-grained control over specific paths in the design, ensuring that critical paths are optimized for performance.
- **Synthesis with strict clock and path constraint:** This synthesis run applies strict constraints on both the clock and specific paths in the design. The synthesis tool is instructed to

optimize the design to meet these strict timing requirements, ensuring that all critical paths are optimized for performance and that the design operates correctly at the specified clock frequency.

- **Synthesis with 0ns clock:** This synthesis run is a special case where the clock period is set to 0ns. This is used to test the design for timing violations and to ensure that the design can handle the worst-case scenarios. It is not a practical synthesis run, but we used it for a lower bound check.

The synthesis process is automated using a TCL script. The full TCL script run by Boh for design compiler is located in the `./scripts/scripts_templates/synthesis_template.tcl`. This script sets up the synthesis environment, applies the necessary constraints, and runs the synthesis tool to generate the gate-level netlist for all the above mentioned cases.

The scripts generates the following files:

- **Area reports:** These reports provide information about the area of the synthesized design, including the total area, area breakdown by functional units, and any unused resources. They are useful for evaluating the efficiency of the design in terms of area utilization.
- **Timing reports:** These reports provide information about the timing performance of the synthesized design, including the critical paths, setup and hold violations, and slack. They are essential for understanding the timing characteristics of the design and ensuring that it meets the specified timing constraints.
- **Power reports:** These reports provide information about the power consumption of the synthesized design, including dynamic and static power. They are useful for evaluating the power efficiency of the design and identifying potential areas for optimization.
- **Gate-level netlist:** This is the output of the synthesis process, which represents the design in terms of gates and flip-flops. It is used for post synthesis analysis and optimization and physical design.
- **Synthesized design constraints:** These are the constraints applied during the synthesis process, which define the target technology, timing requirements, and area constraints. They can be used for further analysis and optimization of the synthesized design.

3.2 Results

Being restricted by the server issues we run less synthesis than we would have liked, but the results are still significant. The synthesis results for the DLX microprocessor are summarized in the following table:

Clock Constraint	Path Constraint	Area	Worst Slack
Unconstrained	Unconstrained	20680.70 μm^2	-
2 ns	Unconstrained	21487.74 μm^2	1.56 ns
2 ns	2 ns	21289.57 μm^2	1.56 ns
1 ns	0.5 ns	21270.15 μm^2	0.57 ns
0.5 ns	0.5 ns	21271.22 μm^2	0.06 ns
0 ns	Unconstrained	21317.50 μm^2	-0.31 ns

Table 3.1: Area and timing results

The area results show that the design has a total area of approximately 20.7 mm² for the unconstrained synthesis, which increases slightly to around 21.5 mm² when a clock constraint is applied.

The area remains relatively stable with the addition of path constraints, indicating that the design is efficient in terms of area utilization. The worst slack values indicate the timing performance of the design. The unconstrained synthesis has no timing constraints, so no slack is reported. When a clock constraint of 2 ns is applied, the design achieves a worst slack of 1.56 ns, indicating that the design can operate at a frequency higher than the specified clock period. As the constraints become stricter, the worst slack decreases, with the 0.5 ns clock and path constraint resulting in a worst slack of only 0.06 ns, indicating that the design is close to meeting the timing requirements.

The synthesis process also includes power consumption analysis, which is crucial for evaluating the design's efficiency and performance.

Clock Constraint	Path Constraint	Leakage Power	Dynamic Power	Total Power
Unconstrained	Unconstrained	334.74 μW	2138.8 μW	2473.6 μW
2 ns	Unconstrained	386.84 μW	4151.2 μW	4538.0 μW
2 ns	2 ns	371.53 μW	4495.0 μW	4866.6 μW
1 ns	0.5 ns	371.49 μW	8990.0 μW	9361.6 μW
0.5 ns	0.5 ns	371.48 μW	17980.1 μW	1835.2 μW
0 ns	Unconstrained	367.26 μW	1355.9 μW	1723.2 μW

Table 3.2: Power results

The power results highlight the trade-off between performance and power consumption. As the constraints become stricter, the synthesis tool optimizes the design to meet tighter timing requirements, which often leads to increased dynamic power consumption. This is because achieving higher performance typically requires faster switching activity and potentially more active resources, which contribute to higher dynamic power. This is normal as at higher frequencies, the design is more active, leading to increased dynamic power consumption. The leakage power, however, remains relatively stable across different synthesis runs, indicating that the design's static power consumption does not vary significantly with the applied constraints. This is an important aspect to consider, as leakage power can be a significant contributor to overall power consumption in modern designs.

Further information about the synthesis can be found in the `./synthesis/dlx_syn` folder, where the synthesis reports and logs are stored. The synthesis results include the area, timing, clock, and power consumption of the synthesized design.

CHAPTER 4

Physical Design

From the synthesis of the DLX microprocessor, we moved to the physical design phase. This phase is crucial as it translates the synthesized netlist into a physical layout that can be fabricated on silicon. The process involves several steps, including floorplanning, placement, clock tree synthesis, routing, and sign-off checks. The physical design of the DLX microprocessor was performed using the Innovus tool, which is part of the Cadence suite. The process was automated using a TCL script, which is run by [Cassiopeia](#). The script sets up the physical design environment, applies the necessary constraints, and runs the Innovus tool to generate the final layout and design files.

From the analysis of the synthesis results we decided to use the strict clock (1ns) and path constraints (0.5ns) for the physical design. This means that the physical design process will be optimized to meet the timing requirements for the specified clock period and critical paths in the design.

The physical design process is automated through [Cassiopeia](#) launched through [Marley](#), which runs the `./scripts/scripts_templates/physical_desing.tcl`. The script sets up the environment, applies the necessary constraints, and runs the Innovus tool to generate the final layout and design files. The script also includes commands to perform various checks and optimizations during the physical design process and generate the corresponding reports. The script is designed based on the laboratory exercises provided by the course, which served as a template for the physical design process. The main steps of the physical design process are as follows:

1. **Floorplanning:** The first step is to define the floorplan of the design, which includes the placement of the core, I/O pads, and power rings. The floorplan is defined in the script using the `setFloorplan` command.
2. **Placement:** The next step is to place the cells in the design. The placement is performed using the `placeDesign` command, which places the cells based on the defined floorplan and constraints.
3. **Clock Tree Synthesis (CTS):** After placement, the clock tree is synthesized to ensure that the clock signal is distributed evenly across the design. This is done using the `clockTreeSynthesis` command.
4. **Routing:** The next step is to route the design, which connects the cells and ensures that all signals are properly routed. This is done using the `routeDesign` command.
5. **Timing analysis and design evaluation:** After routing, the design is analyzed for timing violations and other issues. This is done using the `timeDesign` command, which performs static timing analysis and generates reports on the timing performance of the design.

The physical design process is completed by generating the final layout and design files. The final layout is saved in the `./physical_design` directory, and the reports are generated in the same directory. The reports include timing analysis, area, and other design metrics.

4.1 Results

4.1.1 Area and Gate Count

The physical design of the DLX microprocessor resulted in a total area of 20137.8 μm^2 and a total gate count of 25235. The report shows all the modules, each with its own gate count, cell count, and area. The following table summarizes the module-wise comparison of gates, cells, and area:

Module	Gates	Cells	Area (μm^2)
DLX	25235	8563	20137.8
CU_inst	143	44	114.1
datapath_inst	25089	8517	20021.0
datapath_inst/fetch_inst	508	160	405.9
datapath_inst/decode_inst	14700	4486	11730.9
datapath_inst/decode_inst/decode_RF	13278	3965	10596.1
datapath_inst/decode_inst/add_div_161	105	63	83.8
datapath_inst/execute_inst	2313	1248	1846.0
datapath_inst/execute_inst/r105	236	65	188.3
datapath_inst/execute_inst/r99	190	162	151.9
datapath_inst/execute_inst/srl_123	321	217	256.2
datapath_inst/execute_inst/sll_121	414	224	330.6
datapath_inst/write_mem_rf_inst	7487	2573	5975.2
datapath_inst/write_mem_rf_inst/DRAM_inst	7147	2464	5703.6

Table 4.1: Module-wise comparison of gates, cells, and area

we can see that `datapath\_inst` module is the largest module in terms of gates and area, mostly due to the `decode_RF` which is expected as it is the register file for the decode stage. We can also appreciate that since the alu functions are implemented via VHDL functions extra modules have been created for the different operations, such as `add_div_161`, `srl_123`, and `sll_121`. Further analysis could be done: like swapping the functions with a more complex implementation that could reduce the number of modules and gates or even increase them to reduce timings.

4.1.2 Timing Analysis

The timing analysis was performed at different stages of the physical design process post-clock tree synthesis (CTS) and post-routing. The results are summarized in the following tables.

Setup mode	all	reg2reg	default	Hold mode	all	reg2reg	default
WNS (ns):	0.654	0.654	0.718	WNS (ns):	-0.341	0.075	-0.341
TNS (ns):	0.000	0.000	0.000	TNS (ns):	-245.034	0.000	-245.034
Violating Paths:	0	0	0	Violating Paths:	1308	0	1308
All Paths:	1612	295	1322	All Paths:	1612	295	1322

Table 4.2: timeDesign postCTS setup summary

Table 4.3: timeDesign postCTS hold summary

Setup mode	all	reg2reg	default	Hold mode	all	reg2reg	default
WNS (ns):	0.669	0.669	0.720	WNS (ns):	-0.222	0.073	-0.222
TNS (ns):	0.000	0.000	0.000	TNS (ns):	-71.685	0.000	-71.685
Violating Paths:	0	0	0	Violating Paths:	1198	0	1198
All Paths:	1612	295	1322	All Paths:	1612	295	1322

Table 4.4: timeDesign postRoute setup summary Table 4.5: timeDesign postRoute hold summary

The timing analysis results show that after the clock tree synthesis (CTS), the design meets the setup timing requirements with a WNS of 0.654 ns and no violating paths. However, there are significant hold violations with a WNS of -0.341 ns and 1308 violating paths. This indicates that the design is not meeting the hold timing requirements, which is a critical issue that needs to be addressed. After the routing, the setup timing is improved with a WNS of 0.669 ns and no violating paths, indicating that the design meets the setup timing requirements. However, there are still hold violations with a WNS of -0.222 ns and 1198 violating paths, which means that the design is still not meeting the hold timing requirements.

4.1.3 Conclusion

The results shows that we could have chosen a synthesized design with a less strict clock and path constraints. Due to the server issues we were not able to run the physical design with a less strict clock and path constraints, but we can assume that the results would be better than the one found above.

All the reports are available at [./physical_design](#). We also made a manual version of the physical design available in the folder [./Manual_Physical_Design](#).

CHAPTER 5

Tools and Scripts

The scripts have been developed by the group during the whole semester to help with the course laboratories. The idea was to have a helper script that was not bound to a single specific laboratory, but rather something that requires almost no modification from the user to suits his needs for the needed tasks. The scripts are under heavy development as they were not only useful in the context of the laboratory but also for the DLX and the Verification Workshop. They are fully functional to assist in the simulation, synthesys and physical design of the DLX.

All scripts are better used in a terminal with color support.

All the source code for each script is available at B.

5.1 Docker Container

During the developing of the DLX processor and also the Verification Workshop there were issues with the availability of the servers. To overcome the problem a Docker has been set up with Ubuntu and Intel - ModelSim to allow at least the compilation and simulation with an environment as close as possible as the one on the course servers. To run it, simply install docker and run the command

```
docker run -d -p 2222:22 -p 5901:5901 -p 6080:6080 --name hw_tools ilnerdchuck/hw_tools:full
```

The container can be reached via a VNC web GUI as well using an SSH connection. More information are available on the Github page.

5.2 Marley

This script manages all the other scripts. All functions are pretty straightforward as all the scripts guide the user through the needed configuration via a Command Line Interface (CLI). As the usage is pretty straightforward in the following sections a brief explanation of the script is made and the values used to reproduce our result for the previous section are shown. The only files that the user needs to modify to suits his needs are in the `./scripts/scripts_templates/*` those files are dynamically evaluated to replace some variables placeholders available from the CLI, refer to the source code C for an example. To start it simply run

```
./marley.sh
```

in the root of the DLX folder.

5.2.1 Pupi

The script is written in GO, it basically scrapes all input VHDL files and builds for each file the needed components or packages. Then sweeps the files ordering first the ones without any dependency then as soon as any dependency of the next files are met the file is added to the ordered list. This creates a ordered list of VHDL files to be given to any HDL compiler for analysis.

5.2.2 Briciola

This script automatically calls Pupi to create the `./compile.list` file containing the hierarchy of the VHDL to be compiled or analyzed by Modelsim or Design Compiler. Then it calls the command `vcom` on each file to compile it and add it to the work library

Note: The script will take all the *.vhd files in in the folder and subfolders where marley is called, to remove files from the compilation put them in the `./exclude` folder

5.2.3 Scorzetta

The script allows the user to run the `./scripts/scripts_templates/simulation_template.tcl` file to run the simulation of a given Entity. The script is configured to setup the `./simulations` folder where a subfolder with the name of the current simulation will be created and inside it all simulation output files will be placed.

For the DLX we added the prompt to compile an assembly file from the `./asm` folder.

For our simulation we used:

- the `test` assembly file
- name for the simulation `dlx_sim`
- a resolution of `1ns`
- depending on the assembly code length `300ns` of runtime
- the top level is `tb_dlx`

5.2.4 Boh

The script allows the user to run the `./scripts/scripts_templates/synthesis_template.tcl` file to run the synthesis of a given Entity. The script is configured to setup the `./synthesis` folder where a subfolder with the name of the current synthesis will be created and inside it all reports files will be placed.

For our synthesis we used:

- name for the synthesis `dlx_syn`
- and the top level is `dlx`

5.2.5 Cassiopeia

The script allows the user to run the `./scripts/scripts_templates/physical_design.tcl` file to run the physical design of a given Entity. The script is configured to setup the `./physical_design` folder where all reports files will be placed.

For our physical design we used:

- name for the synthesis `dlx_syn`, as the one used in Boh
- and the top level is `dlx`

CHAPTER 6

Conclusions

The DLX microprocessor project has been a significant undertaking that has allowed us to apply theoretical knowledge in a practical context. The design and implementation of the DLX architecture have provided a comprehensive understanding of microprocessor design, from the Control Unit and Datapath to the synthesis and physical design processes. The project has highlighted the importance of a well-structured design process, including the use of VHDL for hardware description, simulation for verification, and synthesis for implementation. The challenges faced during the project, such as ensuring correct timing and functionality, have reinforced the need for meticulous testing and validation at each stage of the design. The use of custom scripts, such as Marley, Pupi, Briciola, Scorzetta, Boh, and Cassiopeia, has improved the development process, allowing for efficient management of the design workflow. These tools have facilitated the compilation, simulation, synthesis, and physical design tasks, demonstrating the value of automation in hardware design.

6.1 Further Work

Future work could focus on enhancing the DLX microprocessor design by exploring advanced features such as out-of-order execution, dynamic advanced branch prediction, and cache memory implementation. Additionally, further optimization of the synthesis and physical design processes could lead to improved performance and reduced power consumption, like seen in the Synthesis and Optimization course we could apply a multi-Vth algorithm to swap high leakage cells with lower trading a decrease the power consumption with slower cell timing. Changing the encoding of the instructions to a more compact format could also be considered, which would reduce the number of control words needed, thus reducing the overall memory. Regarding the scripts, we could improve the user experience by adding more features, such as integration with other design tools. Furthermore, to enhance the readability of waveforms generated during simulation, for example by using scripts like vcd2wavedrom, could facilitate better analysis and debugging of the design, and improve the quality of the documentation.

APPENDIX A

DLX VHDL Code

A.1 Control unit

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.myTypes.all;
=====

entity control_unit is
  generic (
    MICROCODE_MEM_SIZE : integer := 10; -- Microcode Memory Size
    FUN_SIZE            : integer := 11; -- Func Field Size for R-Type Ops
    OP_CODE_SIZE        : integer := 6;  -- Op Code Size
    IR_SIZE              : integer := 32; -- Instruction Register Size
    CW_SIZE              : integer := 15  -- Control Word Size
  );
  port (
    clk      : in std_logic; -- Clock
    reset    : in std_logic; -- Reset:Active-Low

    -- Instruction Register
    IR_IN : in std_logic_vector(IR_SIZE - 1 downto 0);

    fetch_enable      : out std_logic; -- Fetch Enable Signal
    decode_enable     : out std_logic; -- Decode Enable Signal
    execute_enable    : out std_logic; -- Execute Enable Signal
    memory_enable     : out std_logic; -- Memory Enable Signal
    write_back_enable : out std_logic; -- Write Back Enable Signal

    -- IF Control Signal
    RF_EN   : out std_logic; -- Instruction Register Latch Enable
    RF_re_1 : out std_logic; -- Instruction Register 1 Read Enable
    RF_re_2 : out std_logic; -- Instruction Register 2 Read Enable
    RF_we   : out std_logic; -- Instruction Register Write Enable
```

```

source_select_1 : out std_logic; -- Source Select Signal 1 for ALU
source_select_2 : out std_logic; -- Source Select Signal 2 for ALU
ALU_op          : out std_logic_vector(FUN_SIZE - 1 downto 0); -- ALU Operation
    ↪ Code
DRAM_enable     : out std_logic; -- DRAM Enable Signal
DRAM_WE         : out std_logic; -- DRAM Write Enable Signal
DRAM_RE         : out std_logic; -- DRAM Read Enable Signal
source_select_3 : out std_logic; -- Source Select Signal for Memory Stage ->
    ↪ Write back Stage

inst_type_out : out std_logic_vector(1 downto 0); -- Instruction Type (R, I, J)
opcode_out    : out std_logic_vector(OP_CODE_SIZE - 1 downto 0) -- OpCode of the
    ↪ Instruction
);
end control_unit;
architecture Behavioral of control_unit is
    type cw_memory_Type is array(0 to 43) of std_logic_vector(CW_SIZE - 1 downto 0);
    -- This is the cw_memory table with all the CWs
    signal cw_memory : cw_memory_Type := (
        "111111001000011", -- op: 0x00 R Type
        "100000000000000", -- op: 0x01 not implemented
        "110001110000000", -- op: 0x02 JTYPE_JMP
        "110001111000011", -- op: 0x03 JTYPE_JAL
        "111101000000000", -- op: 0x04 JTYPE_BNEZ
        "111101000000000", -- op: 0x05 JTYPE_BEQZ
        "100000000000000", -- op: 0x06 not implemented
        "100000000000000", -- op: 0x07 not implemented
        "111101011000011", -- op: 0x08 ADDI
        "100000000000000", -- op: 0x09 not implemented
        "111101011000011", -- op: 0x0A SUBI
        "100000000000000", -- op: 0x0B not implemented
        "111101011000011", -- op: 0x0C ANDI
        "111101011000011", -- op: 0x0D ORI
        "111101011000011", -- op: 0x0E XORI
        "100000000000000", -- op: 0x0F not implemented
        "100000000000000", -- op: 0x10 not implemented
        "100000000000000", -- op: 0x11 not implemented
        "100000000000000", -- op: 0x12 not implemented
        "100000000000000", -- op: 0x13 not implemented
        "111101011000011", -- op: 0x14 SLLI
        "100000000000000", -- op: 0x15 NOP
        "111101011000011", -- op: 0x16 SRLI
        "100000000000000", -- op: 0x17 not implemented
        "100000000000000", -- op: 0x18 not implemented
        "111101011000011", -- op: 0x19 SNEI
        "100000000000000", -- op: 0x1A not implemented
        "100000000000000", -- op: 0x1B not implemented
        "111101011000011", -- op: 0x1C SLEI
    );

```

```

"111101011000011", -- op: 0x1D SGEI
"100000000000000", -- op: 0x1E not implemented
"100000000000000", -- op: 0x1F not implemented
"100000000000000", -- op: 0x20 not implemented
"100000000000000", -- op: 0x21 not implemented
"100000000000000", -- op: 0x22 not implemented
"111101011110111", -- op: 0x23 ITYPE_LW
"100000000000000", -- op: 0x24 not implemented
"100000000000000", -- op: 0x25 not implemented
"100000000000000", -- op: 0x26 not implemented
"100000000000000", -- op: 0x27 not implemented
"100000000000000", -- op: 0x28 not implemented
"100000000000000", -- op: 0x29 not implemented
"100000000000000", -- op: 0x2A not implemented
"111101011101000" -- op: 0x2B ITYPE_SW
);

signal opcode      : std_logic_vector(OP_CODE_SIZE - 1 downto 0); -- OpCode of the
    ↳ Instruction
signal cw_target   : std_logic_vector(CW_SIZE - 1 downto 0); -- Control Word for the
    ↳ decode stage
signal cw_ex       : std_logic_vector(CW_SIZE - 1 downto 0); -- Control Word for the
    ↳ execute stage
signal cw_mem      : std_logic_vector(CW_SIZE - 1 downto 0); -- Control Word for the
    ↳ memory stage
signal cw_wb       : std_logic_vector(CW_SIZE - 1 downto 0); -- Control Word for the
    ↳ write back stage

signal opcode_ex   : std_logic_vector(OP_CODE_SIZE - 1 downto 0); -- OpCode for the
    ↳ execute stage
signal opcode_mem   : std_logic_vector(OP_CODE_SIZE - 1 downto 0); -- OpCode for the
    ↳ memory stage
signal opcode_wb    : std_logic_vector(OP_CODE_SIZE - 1 downto 0); -- OpCode for the
    ↳ write back stage

signal alu_func_type : std_logic_vector(FUN_SIZE - 1 downto 0); -- ALU Function
    ↳ Type for the execute stage

begin
opcode <= IR_IN(IR_SIZE - 1 downto IR_SIZE - OP_CODE_SIZE) when reset = '0' else
    TYPE_NOP; -- Extracting opcode from instruction register @reset default to NOP
cw_target <= cw_memory(to_integer(unsigned(opcode))) when reset = '0' else
    (others => '0'); -- Extracting control word from the microcode memory based on
    ↳ the opcode @reset default to NOP

-- need to output the opcode to the datapath for the forwarding unit to check for
    ↳ hazards for the execution stage
opcode_out <= opcode_mem; -- used to stall if the instruction is a load and the
    ↳ source register matches the destination register of the EX stage

```

```

ALU_op      <= alu_func_type; -- ALU operation code output

process (clk)
begin
    if (rising_edge(clk)) then
        if (reset = '1') then
            -- Reset all control words and opcode signals to a NOP filled pipeline
            cw_ex      <= (others => '0');
            cw_mem      <= (others => '0');
            cw_wb      <= (others => '0');
            opcode_ex   <= (others => '0');
            opcode_mem   <= (others => '0');
            opcode_wb   <= (others => '0');
            alu_func_type <= (others => '0');

        else

            -- update control words for each stage
            cw_ex <= cw_target;
            cw_mem <= cw_ex;
            cw_wb <= cw_mem;

            -- Extracting opcode for the next stages
            opcode_ex <= opcode;
            opcode_mem <= opcode_ex;
            opcode_wb <= opcode_mem;

            -- ALU function for the Execute stage
            if opcode = TYPE_NOP then
                alu_func_type <= (others => '0'); -- NOP operation
            elsif opcode = ITYPE_ADDI then
                alu_func_type <= RTYPE_ADD; -- ADDI operation
            elsif opcode = ITYPE_SUBI then
                alu_func_type <= RTYPE_SUB; -- SUBI operation
            elsif opcode = ITYPE_ANDI then
                alu_func_type <= RTYPE_AND; -- ANDI operation
            elsif opcode = ITYPE_ORI then
                alu_func_type <= RTYPE_OR; -- ORI operation
            elsif opcode = ITYPE_XORI then
                alu_func_type <= RTYPE_XOR; -- XORI operation
            elsif opcode = ITYPE_SLLI then
                alu_func_type <= RTYPE_SLL; -- SLLI operation
            elsif opcode = ITYPE_SRLI then
                alu_func_type <= RTYPE_SRL; -- SRLI operation
            elsif opcode = ITYPE_SNEI then
                alu_func_type <= RTYPE_SNE; -- SNEI operation
            elsif opcode = ITYPE_SLEI then
                alu_func_type <= RTYPE_SLE; -- SLEI operation
            elsif opcode = ITYPE_SGEI then

```

```

    alu_func_type <= RTYPE_SGE; -- SGEI operation
elsif opcode = ITYPE_LW then
    alu_func_type <= RTYPE_ADD; -- LW operation, ALU adds offset to base
    ↪ address
elsif opcode = ITYPE_SW then
    alu_func_type <= RTYPE_ADD; -- SW operation, ALU adds offset to base
    ↪ address
elsif opcode = JTYPE_JMP then
    alu_func_type <= RTYPE_ADD; -- JMP operation, ALU adds offset to base
    ↪ address
elsif opcode = JTYPE_JAL then
    alu_func_type <= RTYPE_ADD; -- JAL operation, ALU adds offset to base
    ↪ address
elsif opcode = JTYPE_BEQZ then
    alu_func_type <= RTYPE_ADD; -- BEQZ operation, ALU adds offset to base
    ↪ address
elsif opcode = JTYPE_BNEZ then
    alu_func_type <= RTYPE_ADD; -- BNEZ operation, ALU adds offset to base
    ↪ address
else
    alu_func_type <= (others => '0'); -- Default case, reset ALU function type
    ↪
end if;

-- ALU function for the Execute stage
-- case opcode is
--   when TYPE_NOP =>
--       alu_func_type <= (others => '0'); -- NOP operation
--       opcode_ex      <= (others => '0'); -- NOP operation
--       opcode_mem      <= (others => '0'); -- NOP operation
--       opcode_wb       <= (others => '0'); -- NOP operation

--   when RTYPE =>
--       -- R-Type operation the function is in the IR register
--       alu_func_type <= IR_in(FUN_SIZE - 1 downto 0); -- R-Type operation
--   when ITYPE_ADDI =>
--       alu_func_type <= RTYPE_ADD; -- ADDI1 operation
--   when ITYPE_SUBI =>
--       alu_func_type <= RTYPE_SUB; -- SUBI1 operation
--   when ITYPE_ANDI =>
--       alu_func_type <= RTYPE_AND; -- ANDI1 operation
--   when ITYPE_ORI =>
--       alu_func_type <= RTYPE_OR; -- ORI1 operation
--   when ITYPE_XORI =>
--       alu_func_type <= RTYPE_XOR; -- ORI1 operation
--   when ITYPE_SLLI =>
--       alu_func_type <= RTYPE_SLL; -- ORI1 operation
--   when ITYPE_SRLI =>
--       alu_func_type <= RTYPE_SRL; -- ORI1 operation

```

```

-- when ITYPE_SNEI =>
--     alu_func_type <= RTYPE_SNE; -- ORI1 operation
-- when ITYPE_SLEI =>
--     alu_func_type <= RTYPE_SLE; -- ORI1 operation
-- when ITYPE_SGEI =>
--     alu_func_type <= RTYPE_SGE; -- ORI1 operation
-- when ITYPE_LW =>
--     alu_func_type <= RTYPE_ADD; -- ORI1 operation
-- when ITYPE_SW =>
--     alu_func_type <= RTYPE_ADD; -- ORI1 operation
-- when JTYPE_JMP =>
--     alu_func_type <= RTYPE_ADD; -- ORI1 operation
-- when JTYPE_JAL =>
--     alu_func_type <= RTYPE_ADD; -- ORI1 operation
-- when JTYPE_BEQZ =>
--     alu_func_type <= RTYPE_ADD; -- ORI1 operation
-- when JTYPE_BNEZ =>
--     alu_func_type <= RTYPE_ADD; -- ORI1 operation
-- when others
--     =>
--     alu_func_type <= (others => '0');
-- end case;
end if;

end if;
end process;

-- Instruction type update mostly for the jump and branch instructions
process (opcode, reset)
begin
    if reset = '1' then
        inst_type_out <= (others => '0'); -- NOP operation
    else
        if opcode = TYPE_NOP then
            inst_type_out <= (others => '0'); -- NOP operation
        elsif opcode = ITYPE_ADDI then
            inst_type_out <= "00"; -- ADDI operation
        elsif opcode = ITYPE_SUBI then
            inst_type_out <= "01"; -- SUBI operation
        elsif opcode = ITYPE_ANDI then
            inst_type_out <= "01"; -- ANDI operation
        elsif opcode = ITYPE_ORI then
            inst_type_out <= "01"; -- ORI operation
        elsif opcode = ITYPE_XORI then
            inst_type_out <= "01"; -- XORI operation
        elsif opcode = ITYPE_SLLI then
            inst_type_out <= "01"; -- SLLI operation
        elsif opcode = ITYPE_SRLI then
            inst_type_out <= "01"; -- SRLI operation
        elsif opcode = ITYPE_SNEI then

```

```

    inst_type_out <= "01"; -- SNEI operation
elsif opcode = ITYPE_SLEI then
    inst_type_out <= "01"; -- SLEI operation
elsif opcode = ITYPE_SGEI then
    inst_type_out <= "01"; -- SGEI operation
elsif opcode = ITYPE_LW then
    inst_type_out <= "01"; -- LW operation, ALU adds offset to base address
elsif opcode = ITYPE_SW then
    inst_type_out <= "01"; -- SW operation, ALU adds offset to base address
elsif opcode = JTYPE_JMP then
    inst_type_out <= "11"; -- JMP operation, ALU adds offset to base address
elsif opcode = JTYPE_JAL then
    inst_type_out <= "11"; -- JAL operation, ALU adds offset to base address
elsif opcode = JTYPE_BEQZ then
    inst_type_out <= "01"; -- BEQZ operation, ALU adds offset to base address
elsif opcode = JTYPE_BNEZ then
    inst_type_out <= "01"; -- BNEZ operation, ALU adds offset to base address
else
    inst_type_out <= (others => '0'); -- Default case, reset ALU function type
end if;

-- case opcode is
--   when TYPE_NOP          =>
--       inst_type_out <= (others => '0'); -- NOP operation
--   when RTYPE             =>
--       inst_type_out <= "00"; -- R-Type operation
--   when ITYPE_ADDI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SUBI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_ANDI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_ORI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_XORI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SLLI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SRLI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SNEI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SLEI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SGEI =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_LW =>
--       inst_type_out <= "01"; -- I-Type operation
--   when ITYPE_SW =>

```

```

--      inst_type_out <= "01"; -- I-Type operation
--      when JTYPE_JMP =>
--      inst_type_out <= "11"; -- J-Type operation
--      when JTYPE_JAL =>
--      inst_type_out <= "11"; -- J-Type operation but also I-Type operation
--      for the write back stage
--      when JTYPE_BEQZ =>
--      inst_type_out <= "01"; -- J-Type operation
--      when JTYPE_BNEZ =>
--      inst_type_out <= "01"; -- J-Type operation
--      when others =>
--      inst_type_out <= "00"; -- J-Type operation
--      end case;
end if;
end process;

-- Update Control signals for the different stages of the pipeline
-- Fetch stage
fetch_enable <= cw_target(14); -- Fetch Enable Signal
-- Decode stage
decode_enable <= cw_target(13); -- Decode Enable Signal
RF_EN        <= cw_wb(0) or cw_target(12); -- Register file Enable Signal either
--      form the Write Back stage or Decode stage
RF_re_1      <= cw_target(11); -- Instruction Register 1 Read Enable
RF_re_2      <= cw_target(10); -- Instruction Register 2 Read Enable
-- Execute stage
execute_enable <= cw_ex(9); -- Execute Enable Signal
source_select_1 <= cw_ex(8); -- Source Select Signal 1 for ALU
source_select_2 <= cw_ex(7); -- Source Select Signal 1 for ALU
-- Memory stage
memory_enable <= cw_mem(6); -- Memory Enable Signal
DRAM_enable   <= cw_mem(5); -- DRAM Enable Signal
DRAM_RE       <= cw_mem(4); -- DRAM Read Enable Signal
DRAM_WE       <= cw_mem(3); -- DRAM Write Enable Signal
source_select_3 <= cw_mem(2); -- Source Select Signal for Memory Stage
-- Write Back stage
write_back_enable <= cw_wb(1); -- Write Back Stage Enable
RF_we          <= cw_wb(1); -- Instruction Register Write Enable
end architecture;

```

A.2 DataPath

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.math_real.all;
=====
-- Entity Declaration
entity datapath is

```

```

generic (
  I_SIZE      : integer := 32; -- Size of instruction in bits
  OPERAND_SIZE : integer := 32; -- Size operands in bits
  NUM_REG     : integer := 5;  -- Number of registers in bits (log2(RF_SIZE))
  FUN_SIZE    : integer := 11 -- ALU operation code size in bits
);
port (
  -- DLX Processor signals
  clk   : in std_logic; -- Clock signal
  reset : in std_logic; -- Reset signal

  -- Control unit signals
  fetch_enable   : in std_logic; -- Fetch stage enable signal
  decode_enable  : in std_logic; -- Decode stage enable signal
  execute_enable : in std_logic; -- Execute stage enable signal
  memory_enable  : in std_logic; -- Memory stage enable signal
  write_back_enable : in std_logic; -- Write Back stage enable signal

  -- Decode stage signals
  -- Register file signals
  RF_en : in std_logic; -- Register file enable signal
  -- Operand 1
  RF_re_1 : in std_logic; -- Register file read enable for source 1 register
  -- Operand 2
  RF_re_2 : in std_logic; -- Register file read enable for source 2 register
  -- Register file write signals
  RF_we : in std_logic; -- Register file write enable

  -- Execute stage signals
  source_1_select : in std_logic; -- Source 1 select signal
  source_2_select : in std_logic; -- Source 2 select signal
  ALU_op          : in std_logic_vector(FUN_SIZE - 1 downto 0); -- ALU operation
  ↪ code input

  -- Memory stage signals
  DRAM_enable : in std_logic; -- DRAM enable signal
  DRAM_RE     : in std_logic; -- DRAM read enable signal
  DRAM_WE     : in std_logic; -- DRAM write enable signal
  source_3_select : in std_logic; -- Source select signal for memory stage
  ↪ (MEM/ALU) -> Could be replaced by DRAM_enable

  inst_type : in std_logic_vector(1 downto 0); -- Instruction type (R-type,
  ↪ I-type, J-type)
  opcode    : in std_logic_vector(5 downto 0); -- Opcode of the instruction

  IR_out : out std_logic_vector(I_SIZE - 1 downto 0) -- Instruction register (for
  ↪ the CU)
);
end entity datapath;

```

```

=====
-- Architecture Declaration
architecture rtl of datapath is

    component fetch_stage is
        generic (
            OPERAND_SIZE : integer := OPERAND_SIZE; -- Size operands in bits
            I_SIZE       : integer := I_SIZE -- Size of instruction in bits
        );
        port (
            -- DLX Processor signals
            clk      : in std_logic; -- Clock signal
            reset    : in std_logic; -- Reset signal
            -- Control unit signals
            stage_enable : in std_logic; -- Program Counter enable signal

            -- Input/Output signals
            next_PC_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output
            --   Program Counter
            IR_out      : out std_logic_vector(I_SIZE - 1 downto 0); -- Output Instruction
            --   Register
            -- Jump/Branch signals
            jmp_en      : in std_logic; -- Jump enable signal
            jmp_addr    : in std_logic_vector(I_SIZE - 1 downto 0) -- Jump address input
        );
    end component fetch_stage;

    component decode_stage is
        generic (
            I_SIZE       : integer := 32; -- Size of instruction in bits
            OPERAND_SIZE : integer := 32; -- Size operands in bits
            NUM_REG      : integer := 5 -- Number of registers in bits (log2(RF_SIZE))
        );
        port (
            -- DLX Processor signals
            clk      : in std_logic; -- Clock signal
            reset    : in std_logic; -- Reset signal

            -- Control unit signals
            stage_enable : in std_logic; -- Output registers enable signal
            -- Register file signals
            RF_en       : in std_logic; -- Register file enable signal
            -- Operand 1
            RF_re_1     : in std_logic; -- Register file read enable for source 1 register
            -- Operand 2
            RF_re_2     : in std_logic; -- Register file read enable for source 2 register
            -- Register file write signals
            RF_we       : in std_logic; -- Register file write enable

```

```

inst_type : in std_logic_vector(1 downto 0); -- Instruction type (R-type,
↳ I-type, J-type)

-- Input/Output signals
IR_in : in std_logic_vector(I_SIZE - 1 downto 0); -- Instruction register
PC_in : in std_logic_vector(I_SIZE - 1 downto 0); -- Program Counter

PC_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate value
↳ for source 1
A : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output for
↳ source 1 operand
B : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output for
↳ source 2 operand
IMM_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate value
↳ for source 1
-- Register file write address and data ( Write Back stage)
RD_addr_in : in std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
↳ address for destination register (used in Write Back stage)
RD_data_in : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Register file
↳ data for the destination register (used in Write Back stage)
-- Register file read addresses for source registers ( Used for Forwarding)
RD_out : out std_logic_vector(NUM_REG - 1 downto 0); -- Register file
↳ write address for destination register bypass (not used in decode stage)
R_source_1 : out std_logic_vector(NUM_REG - 1 downto 0); -- Register 1 source
↳ address for forwarding
R_source_2 : out std_logic_vector(NUM_REG - 1 downto 0); -- Register 2 source
↳ address for forwarding

-- Signals to handle jump instructions
jmp_en : out std_logic; -- Jump enable signal
jmp_addr : out std_logic_vector(OPERAND_SIZE - 1 downto 0) -- Jump address
↳ output
);
end component decode_stage;

component execute_stage is
generic (
    OPERAND_SIZE : integer := 32; -- Size operands in bits
    NUM_REG : integer := 5; -- Number of registers in bits (log_2(RF_SIZE))
    FUN_SIZE : integer := 11 -- Size of ALU operation code in bits
);
port (
    -- DLX Processor signals
    clk : in std_logic; -- Clock signal
    reset : in std_logic; -- Reset signal

    -- Control unit signals
    stage_enable : in std_logic; -- ALU enable signal

```

```

-- ALU operation signals
alu_op      : in std_logic_vector(FUN_SIZE - 1 downto 0); -- ALU operation
  ↳ code input
source_1_select : in std_logic; -- Source 1 select signal
source_2_select : in std_logic; -- Source 2 select signal

-- Input/Output signals
PC_in  : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- PC value input
  ↳ (IMM_1)
A      : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Operand A input
B      : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Operand B input
IMM_in : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate 2 value
  ↳ input

alu_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- ALU result
  ↳ output

MEM_data_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory data
  ↳ output (for memory operations)
-- Register file signals
RD_in  : in std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
  ↳ address for destination register input (Bypass)
RD_out : out std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
  ↳ address for destination register output (Bypass)

-- Forwarding signals
-- 00: no forwarding, 01: forward from MEM stage, 11: forward from WB stage
fwd_A : in std_logic_vector(1 downto 0); -- Forwarded value for source A
  ↳ operand
fwd_B : in std_logic_vector(1 downto 0); -- Forwarded value for source B
  ↳ operand

RS_wb : in std_logic_vector(OPERAND_SIZE - 1 downto 0) -- Data Forwarded from
  ↳ Write Back stage (WB stage)
);
end component execute_stage;
component write_mem_rf_stage is
generic (
  OPERAND_SIZE : integer := 32; -- Size operands in bits
  NUM_REG      : integer := 5 -- Number of registers in bits (log_2(RF_SIZE))
);
port (
  -- DLX Processor signals
  clk  : in std_logic; -- Clock signal
  reset : in std_logic; -- Reset signal
  -- Control unit signals
  stage_enable : in std_logic; -- stage enable
  DRAM_enable  : in std_logic; -- DRAM enable signal
  DRAM_RE      : in std_logic; -- DRAM read enable signal

```

```

DRAM_WE      : in std_logic; -- DRAM write enable signal
source_select : in std_logic; -- Source select signal MEM/ALU
-- Input/Output signals
alu_result    : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- ALU result
↳ input
MEM_data_in   : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory data
↳ input (from execute stage)
MEM_stage_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory
↳ stage output -> to Write Back stage

-- Register file signals
RD_in  : in std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
↳ address for destination register input
RD_out : out std_logic_vector(NUM_REG - 1 downto 0) -- Register file address
↳ input (WB stage)
);
end component write_mem_rf_stage;

component forwarding_unit is
generic (
  OPERAND_SIZE : integer := 32; -- Size operands in bits
  NUM_REG       : integer := 5 -- Number of registers in bits (log_2(RF_SIZE))
);
port (
  -- Control unit signals
  stage_enable : in std_logic; -- Stage enable signal

  -- Input/Output signals
  EX_MEM_RD  : in std_logic_vector(NUM_REG - 1 downto 0); -- Destination
↳ register from EX stage
  MEM_WB_RD  : in std_logic_vector(NUM_REG - 1 downto 0); -- Destination
↳ register from MEM stage
  EX_MEM_RegW : in std_logic; -- Register write enable from MEM stage
  MEM_WB_RegW : in std_logic; -- Register write enable from WB stage
  ID_RS_1     : in std_logic_vector(NUM_REG - 1 downto 0); -- Source register 1
↳ from EX stage
  ID_RS_2     : in std_logic_vector(NUM_REG - 1 downto 0); -- Source register 2
↳ from EX stage
  opcode      : in std_logic_vector(5 downto 0); -- Opcode of the instruction
  fwd_A       : out std_logic_vector(1 downto 0); -- Forwarding control for
↳ source A
  fwd_B       : out std_logic_vector(1 downto 0); -- Forwarding control for
↳ source B
  stall       : out std_logic -- Stall signal to control data hazards
);
end component forwarding_unit;

-- Signals for the fetch stage

```

```

signal PC_FD : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Program Counter
    ↳ output Fetch -> Decode stage
signal PC_DE : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Program Counter
    ↳ output Decode -> Execute stage
signal IR : std_logic_vector(I_SIZE - 1 downto 0); -- Instruction Register
    ↳ output

-- Signals for jumps
signal jmp_en : std_logic; -- Jump enable signal
signal jmp_addr : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Jump address
    ↳ input

-- Signals for the decode stage
signal A : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output for
    ↳ source 1 operand
signal B : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output for
    ↳ source 2 operand
signal IMM_out : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate
    ↳ value for source 1 output
signal RD_DE : std_logic_vector(NUM_REG - 1 downto 0); -- Register file
    ↳ write address for destination register input Decode -> Execute stage
signal RD_addr_WBD : std_logic_vector(NUM_REG - 1 downto 0); -- Register file
    ↳ write address for destination register input Write Back -> Decode (used in
    ↳ Write Back stage)
signal RD_data_WBD : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Register file
    ↳ write address for destination register Write Back -> Decode (used in Write
    ↳ Back stage)
signal R_source_1 : std_logic_vector(NUM_REG - 1 downto 0); -- Register file
    ↳ write address for destination register Decode -> Execute (for forwarding)
signal R_source_2 : std_logic_vector(NUM_REG - 1 downto 0); -- Register file
    ↳ write address for destination register Decode -> Execute (for forwarding)
-- Signals for the execute stage
signal alu_out : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- ALU result
    ↳ output
signal MEM_data_out : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory data
    ↳ output (data for memory operations)
signal RD_EM : std_logic_vector(NUM_REG - 1 downto 0); -- Register file
    ↳ write address for destination register input Decode -> Execute stage

-- Forwarding signals
signal fwd_A : std_logic_vector(1 downto 0); -- Forwarded value for source A
    ↳ operand
signal fwd_B : std_logic_vector(1 downto 0); -- Forwarded value for source B
    ↳ operand
signal stall : std_logic; -- Stall signal to control data hazards

signal f_en : std_logic; -- True Fetch stage enable signal
signal d_en : std_logic; -- True Decode stage enable signal

```

```

begin
  f_en <= fetch_enable and stall; -- Fetch stage enable signal disabled if stall is
    ↪ needed
  d_en <= decode_enable and stall; -- Fetch stage enable signal disabled if stall is
    ↪ needed

  -- Instantiate the fetch stage
  fetch_inst : fetch_stage
  generic map(
    OPERAND_SIZE => OPERAND_SIZE,
    I_SIZE       => I_SIZE
  )
  port map
  (
    clk          => clk,
    reset        => reset,
    stage_enable => f_en,
    next_PC_out  => PC_FD, -- Output Program Counter
    IR_out       => IR, -- Output Instruction Register
    jmp_en       => jmp_en, -- Jump enable signal
    jmp_addr     => jmp_addr -- Jump address input
  );

  -- Instantiate the decode stage
  decode_inst : decode_stage
  generic map(
    I_SIZE       => I_SIZE,
    OPERAND_SIZE => OPERAND_SIZE,
    NUM_REG      => NUM_REG
  )
  port map
  (
    clk          => clk,
    reset        => reset,
    stage_enable => d_en,
    RF_en        => RF_en, -- Register file enable signal
    RF_re_1      => RF_re_1, -- Register file read enable for source 1 register
    RF_re_2      => RF_re_2, -- Register file read enable for source 2 register
    RF_we        => RF_we, -- Register file write enable
    inst_type     => inst_type, -- Instruction type (R-type, I-type, J-type)
    IR_in        => IR, -- Instruction register input
    PC_in        => PC_FD, -- Program Counter input

    PC_out       => PC_DE, -- Immediate value for source 1 output
    A            => A, -- Output for source 1 operand
    B            => B, -- Output for source 2 operand
    IMM_out      => IMM_out, -- Immediate value for source 1 output
  )
end

```

```

RD_addr_in => RD_addr_WBD, -- Register file write address for destination
↳ register input (used in Write Back stage)
RD_data_in => RD_data_WBD, -- Register file data for the destination register
↳ write (used in Write Back stage)

RD_out      => RD_DE, -- Register file write address for destination register
↳ bypass (not used in decode stage)
R_source_1 => R_source_1, -- Register file write address for destination
↳ register bypass (not used in decode stage)
R_source_2 => R_source_2, -- Register file write address for destination
↳ register bypass (not used in decode stage)

jmp_en      => jmp_en, -- Jump enable signal
jmp_addr    => jmp_addr -- Jump address output
);
-- Instantiate the execute stage
execute_inst : execute_stage
generic map(
  OPERAND_SIZE => OPERAND_SIZE,
  NUM_REG      => NUM_REG,
  FUN_SIZE     => FUN_SIZE
)
port map
(
  clk          => clk,
  reset        => reset,
  stage_enable => execute_enable, -- ALU enable signal

  alu_op        => ALU_op, -- ALU operation code input
  source_1_select => source_1_select, -- Source 1 select signal
  source_2_select => source_2_select, -- Source 2 select signal

  PC_in  => PC_DE, -- Immediate 1 value input
  A      => A, -- Operand A input
  B      => B, -- Operand B input
  IMM_in => IMM_out, -- Immediate 2 value input

  alu_out      => alu_out, -- ALU result output
  MEM_data_out => MEM_data_out, -- Memory data output (data for memory
↳ operations)

  RD_in  => RD_DE, -- Register file write address for destination register input
  RD_out => RD_EM, -- Register file write address for destination register output
↳ (conenct directly to decode stage)

  fwd_A => fwd_A, -- Forwarded value for source A operand (not used in this
↳ example)
  fwd_B => fwd_B, -- Forwarded value for source B operand (not used in this
↳ example)

```

```

    RS_wb => RD_data_WBD -- Data Forwarded from Write Back stage (WB stage)
);

-- Instantiate the write memory and write back file stage
write_mem_rf_inst : write_mem_rf_stage
generic map(
    OPERAND_SIZE => OPERAND_SIZE,
    NUM_REG      => NUM_REG
)
port map
(
    clk          => clk,
    reset        => reset,
    stage_enable => memory_enable, -- stage enable signal
    DRAM_enable  => DRAM_enable, -- DRAM enable signal
    DRAM_RE      => DRAM_RE, -- DRAM read enable signal
    DRAM_WE      => DRAM_WE, -- DRAM write enable signal
    source_select => source_3_select, -- Source select signal MEM/ALU

    alu_result    => alu_out, -- ALU result input
    MEM_data_in   => MEM_data_out, -- Memory data input (from execute stage)
    MEM_stage_out => RD_data_WBD, -- Memory stage output

    RD_in  => RD_EM, -- Register file write address for destination register input
    RD_out => RD_addr_WBD -- Register file address input (WB stage)
);

-- Instantiate the forwarding unit
forwarding_unit_inst : forwarding_unit
generic map(
    OPERAND_SIZE => OPERAND_SIZE,
    NUM_REG      => NUM_REG
)
port map
(
    stage_enable => execute_enable, -- Stage enable signal

    EX_MEM_RD  => RD_DE, -- Destination register from EX stage
    MEM_WB_RD  => RD_addr_WBD, -- Destination register from MEM stage
    EX_MEM_RegW => RF_we, -- Register write enable from MEM stage
    MEM_WB_RegW => RF_we, -- Register write enable from WB stage
    ID_RS_1    => R_source_1, -- Source register 1 from EX stage
    ID_RS_2    => R_source_2, -- Source register 2 from EX stage
    opcode     => opcode, -- Opcode of the instruction

    fwd_A => fwd_A, -- Forwarding control for source A (not used in this example)
    fwd_B => fwd_B, -- Forwarding control for source B (not used in this example)
    stall => stall -- Stall signal to control data hazards (not used in this
    ↪ example)

```

```

);

IR_out <= IR when reset = '0' else
    X"54000000"; -- Connect the instruction register output to the datapath output
end architecture rtl;
=====

```

A.3 Fetch Stage

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.math_real.all;
use work.myTypes.all;

=====

entity fetch_stage is
    generic (
        OPERAND_SIZE : integer := 32; -- Size operands in bits
        I_SIZE        : integer := 32 -- Size of instruction in bits
    );
    port (
        -- DLX Processor signals
        clk      : in std_logic; -- Clock signal
        reset    : in std_logic; -- Reset signal
        -- Control unit signals
        stage_enable : in std_logic; -- Program Counter enable signal

        -- Input/Output signals
        next_PC_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output Program
        -- Counter
        IR_out      : out std_logic_vector(I_SIZE - 1 downto 0); -- Output Instruction
        -- Register
        -- Jump/Branch signals
        jmp_en      : in std_logic; -- Jump enable signal
        jmp_addr    : in std_logic_vector(I_SIZE - 1 downto 0) -- Jump address input
    );
end fetch_stage;

architecture Behavioral of fetch_stage is
    --Instruction Ram
    component IRAM
        generic (
            RAM_DEPTH : integer := 48; -- Depth of instruction RAM
            I_SIZE     : integer := I_SIZE -- Size of instruction in bits
        );
        port (

```

```

    Rst  : in std_logic;
    Addr : in std_logic_vector(I_SIZE - 1 downto 0);
    Dout : out std_logic_vector(I_SIZE - 1 downto 0)
  );
end component;

signal PC      : std_logic_vector(I_SIZE - 1 downto 0); -- Program Counter
signal IR      : std_logic_vector(I_SIZE - 1 downto 0); -- Instruction
  ↳ Register
signal RAM_data_out : std_logic_vector(I_SIZE - 1 downto 0); -- Instruction
  ↳ Register
begin
  -- Instantiate the IRAM component
  IRAM_inst : entity work.IRAM
    generic map(
      RAM_DEPTH => 48, -- Depth of instruction RAM
      I_SIZE    => I_SIZE -- Size of instruction in bits
    )
    port map
    (
      Rst => reset,
      Addr => PC, -- Address for IRAM
      Dout => RAM_data_out -- Next instruction output
    );
  -- Process to handle Program Counter and Instruction Register
  FETCH_PROCESS : process (clk, reset)
  begin
    if reset = '1' then
      PC <= (others => '0'); -- Reset Program Counter
      IR <= X"54000000"; -- Reset Instruction Register
    elsif rising_edge(clk) then
      if stage_enable = '1' then
        if jmp_en = '1' then
          PC <= std_logic_vector(signed(PC) + signed(jmp_addr)); -- Jump to
            ↳ specified address
          IR <= X"54000000"; -- Force a NOP operation to stall the pipeline
        else
          PC <= std_logic_vector(signed(PC) + 1); -- Increment Program Counter
          IR <= RAM_data_out; -- Load instruction from IRAM
        end if;
      end if;
    end if;
  end if;
end process FETCH_PROCESS;

  -- Output assignments
  next_pc_out <= PC; -- Output Program Counter value
  ir_out      <= IR; -- Output fetched instruction
end Behavioral;

```

A.4 Decode Stage

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.math_real.all;
use work.myTypes.all;
=====
-- Entity Declaration
-- Operation types
-- 00: R-type, 01: I-type, 11: J-type
entity decode_stage is
  generic (
    I_SIZE      : integer := 32; -- Size of instruction in bits
    OPERAND_SIZE : integer := 32; -- Size operands in bits
    NUM_REG     : integer := 5 -- Number of registers in bits (log_2(RF_SIZE))
  );
  port (
    -- DLX Processor signals
    clk      : in std_logic; -- Clock signal
    reset    : in std_logic; -- Reset signal

    -- Control unit signals
    stage_enable : in std_logic; -- Output registers enable signal
    -- Register file signals
    RF_en : in std_logic; -- Register file enable signal
    -- Operand 1
    RF_re_1 : in std_logic; -- Register file read enable for source 1 register
    -- Operand 2
    RF_re_2 : in std_logic; -- Register file read enable for source 2 register
    -- Register file write signals
    RF_we : in std_logic; -- Register file write enable
    inst_type : in std_logic_vector(1 downto 0); -- Instruction type (R-type,
    -- I-type, J-type)

    -- Input/Output signals
    IR_in : in std_logic_vector(I_SIZE - 1 downto 0); -- Instruction register
    PC_in : in std_logic_vector(I_SIZE - 1 downto 0); -- Program Counter

    PC_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- PC output for jump
    -- instructions (IMM_1)
    A : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output for source
    -- 1 operand
    B : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Output for source
    -- 2 operand
    IMM_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate value
    -- for source 2

```

```

-- Register file write address and data ( Write Back stage)
RD_addr_in : in std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
↳ address for destination register (used in Write Back stage)
RD_data_in : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Register file
↳ data for the destination register write (used in Write Back stage)
-- Register file read addresses for source registers ( Used for Forwarding)
RD_out      : out std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
↳ address for destination register bypass (not used in decode stage)
R_source_1  : out std_logic_vector(NUM_REG - 1 downto 0); -- Register 1 source
↳ address for forwarding
R_source_2  : out std_logic_vector(NUM_REG - 1 downto 0); -- Register 2 source
↳ address for forwarding

-- Signals to handle jump instructions
jmp_en      : out std_logic; -- Jump enable signal
jmp_addr    : out std_logic_vector(OPERAND_SIZE - 1 downto 0) -- Jump address
↳ output
);
end decode_stage;

architecture Behavioral of decode_stage is

-- Register file
component register_file is
generic (
    NBit : integer := OPERAND_SIZE; -- number of bit per register
    NReg : integer := 2 ** NUM_REG -- number of memory locations/registers
);
port (
    CLK      : in std_logic; -- Clock Signal
    RESET    : in std_logic; -- Reset Signal
    ENABLE    : in std_logic; -- Enable Signal
    RD1      : in std_logic; -- Read 1 Signal
    RD2      : in std_logic; -- Read 2 Signal
    WR       : in std_logic; -- Write Signal
    ADD_WR   : in std_logic_vector(integer(ceil(log2(real(NReg)))) - 1 downto 0);
↳ -- Write Address
    ADD_RD1  : in std_logic_vector(integer(ceil(log2(real(NReg)))) - 1 downto 0);
↳ -- Read 1 Address
    ADD_RD2  : in std_logic_vector(integer(ceil(log2(real(NReg)))) - 1 downto 0);
↳ -- Read 2 Address
    DATAIN  : in std_logic_vector(NBit - 1 downto 0); -- Data in Signal
    OUT1     : out std_logic_vector(NBit - 1 downto 0); -- Data Out 1 Signal
    OUT2     : out std_logic_vector(NBit - 1 downto 0) -- Data Out 2 Signal
);
end component register_file;

signal opcode : std_logic_vector(5 downto 0); -- Opcode of the instruction
-- Register file read addresses

```

```

-- RF_R1_addr and RF_R2_addr are used to read the source registers
signal RF_R1_addr : std_logic_vector(NUM_REG - 1 downto 0); -- Register file read
↳ address for source 1
signal RF_R2_addr : std_logic_vector(NUM_REG - 1 downto 0); -- Register file read
↳ address for source 2

-- Register File Output operands
signal RF_data_out_1 : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Register
↳ Output for source 1 register
signal RF_data_out_2 : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Register
↳ Output for source 2 register

-- Immediate values for source operands
signal IMM_1 : std_logic_vector(OPERAND_SIZE - 1 - 6 downto 0); -- Immediate
↳ value for source 1 J-Type instruction
signal IMM_2 : std_logic_vector(OPERAND_SIZE - 1 - 16 downto 0); -- Immediate
↳ value for source 2 I-Type instruction
signal IMM_ext : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate value
↳ for source 2 I-Type instruction
begin
  decode_RF : entity work.register_file
    generic map(
      NBit => OPERAND_SIZE, -- Number of bits per register
      NReg => 2 ** NUM_REG -- Number of registers
    )
    port map
    (
      clk      => clk,
      reset    => reset,
      ENABLE   => RF_en, -- Register file enable signal
      RD1      => RF_re_1, -- Read 1 signal for source 1 register
      RD2      => RF_re_2, -- Read 2 signal for source 2 register
      WR       => RF_we, -- Write signal for register file
      ADD_WR   => RD_addr_in, -- Write address for destination register
      ADD_RD1  => RF_R1_addr, -- Read address for source 1 register
      ADD_RD2  => RF_R2_addr, -- Read address for source 2 register
      DATAIN  => RD_data_in, -- Data input for register file (not used in decode
↳ stage)
      OUT1     => RF_data_out_1, -- Data output for source 1 register
      OUT2     => RF_data_out_2 -- Data output for source 2 register
    );
  -- IR signal splitting
  RF_R1_addr <= IR_in(25 downto 21) when reset = '0' else
    (others => '0'); -- Source 1 register address (rs)
  RF_R2_addr <= IR_in(20 downto 16) when reset = '0' else
    (others => '0'); -- Source 2 register address (rt)
  IMM_1 <= IR_in(25 downto 0) when reset = '0' else
    (others => '0'); -- Immediate value for source 1 (J-Type instruction)
  IMM_2 <= IR_in(15 downto 0) when reset = '0' else

```

```

    (others => '0'); -- Immediate value for source 2 (I-Type instruction)
opcode <= IR_in(31 downto 26) when reset = '0' else
    (others => '0'); -- Opcode of the instruction

-- Immediate value extension for I-Type instructions
IMM_ext <= std_logic_vector(resize(signed(IMM_2), IMM_out'length)) when inst_type
    <= "01" else
    std_logic_vector(resize(signed(IMM_1), IMM_out'length)) when inst_type = "11"
    <= else
    (others => '0'); -- Sign-extend immediate value for source 2 (I-Type
    <= instruction)

-- Jump instruction handling
-- If opcode is bgez and A is zero, set jump enable and address

jmp_en <= '1' when (opcode = JTYPE_BEQZ) and RF_data_out_1 = (RF_data_out_1'range
    <= => '0') else -- beqz instruction
    '1' when (opcode = JTYPE_BNEZ) and RF_data_out_1 /= (RF_data_out_1'range
    <= => '0') else -- bnez instruction
    '1' when (opcode = JTYPE_JMP) or (opcode = JTYPE_JAL) else -- jmp and jal
    <= instructions
    '0'; -- Jump enable signal for J-Type instructions

jmp_addr <= std_logic_vector(signed(IMM_ext)/4); -- jump address output

-- Process to decode instruction and read operands from register file
DECODE_PROCESS : process (clk, reset)
begin
    if reset = '1' then
        A <= (others => '0'); -- Reset output for source 1 operand
        B <= (others => '0'); -- Reset output for source 2 operand
        PC_out <= (others => '0'); -- Reset immediate value output for source 1
        IMM_out <= (others => '0'); -- Reset immediate value output for source 2
        RD_out <= (others => '0'); -- Reset register file write address for
        <= destination register
        R_source_1 <= (others => '0'); -- Reset register source 1
        R_source_2 <= (others => '0'); -- Reset register source 2

    elsif rising_edge(clk) then
        if stage_enable = '1' then
            -- Decode instruction and set read addresses based on instruction format
            -- Register values are read from the register file
            A <= RF_data_out_1; -- Source 1 register
            B <= RF_data_out_2; -- Source 2 register

            -- Immediate values
            PC_out <= PC_in; -- PC to compute a JUMP instruction
            R_source_1 <= RF_R1_addr; -- Source 1 register
            R_source_2 <= RF_R2_addr; -- Source 2 register

```

```

-- Decode the address for destination register
if inst_type = "00" then
    RD_out <= IR_in(15 downto 11);
elsif inst_type = "01" then
    RD_out <= IR_in(20 downto 16);
elsif inst_type = "11" then
    if opcode = JTYPE_JAL then
        RD_out <= (others => '1'); -- Link register (R31 or the last one) for
        ↪ JAL instruction
    else
        RD_out <= (others => '0');
    end if;
else
    RD_out <= (others => '0');
end if;
IMM_out <= IMM_ext; -- Immediate value for source 2
end if;
end process DECODE_PROCESS;
end Behavioral;
=====

```

A.5 Execute Stage

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.math_real.all;
use work.myTypes.all;
=====
-- Entity Declaration
entity execute_stage is
    generic (
        OPERAND_SIZE : integer := 32; -- Size operands in bits
        NUM_REG       : integer := 5; -- Number of registers in bits (log2(RF_SIZE))
        FUN_SIZE      : integer := 11 -- ALU function size in bits
    );
    port (
        -- DLX Processor signals
        clk    : in std_logic; -- Clock signal
        reset  : in std_logic; -- Reset signal

        -- Control unit signals
        stage_enable : in std_logic; -- ALU enable signal

        -- ALU operation signals
        alu_op       : in std_logic_vector(FUN_SIZE - 1 downto 0); -- ALU operation
        ↪ code input
    );
end execute_stage;

```

```

source_1_select : in std_logic; -- Source 1 select signal
source_2_select : in std_logic; -- Source 2 select signal

-- Input/Output signals
PC_in  : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- PC input (Program
↳ Counter) (for jump instructions)
A      : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Operand A input
B      : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Operand B input
IMM_in : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Immediate 2 value
↳ input

alu_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- ALU result output

MEM_data_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory data
↳ output (B source bypass for memory operations)
-- Register file signals
RD_in  : in std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
↳ address for destination register input
RD_out : out std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
↳ address for destination register output (conenct directly to decode stage)

-- Forwarding signals
-- 00: no forwarding, 01: forward from MEM stage, 11: forward from WB stage
fwd_A : in std_logic_vector(1 downto 0); -- Forwarded value for source A
↳ operand
fwd_B : in std_logic_vector(1 downto 0); -- Forwarded value for source B
↳ operand

RS_wb : in std_logic_vector(OPERAND_SIZE - 1 downto 0) -- Data Forwarded from
↳ Write Back stage (WB stage)
);
end execute_stage;
=====
architecture Behavioral of execute_stage is

    signal source_1 : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Selected source
    ↳ 1 operand
    signal source_2 : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Selected source
    ↳ 2 operand
    signal a_fw      : std_logic_vector(OPERAND_SIZE - 1 downto 0);
    signal b_fw      : std_logic_vector(OPERAND_SIZE - 1 downto 0);
    signal result     : std_logic_vector(OPERAND_SIZE - 1 downto 0); -- ALU output
    ↳ signal

begin
    -- TODO: Use p4 adder, booth multiplier, and T2 shifter (this if we have time to
    ↳ implement them)

    -- Forwarding logic for source 1 operand

```

```

a_fw <= A when fwd_A = "00" else
  result when fwd_A = "01" else
    RS_wb; -- Forwarded value for source 1 operand

-- Forwarding logic for source 2 operand
b_fw <= B when fwd_B = "00" else
  result when fwd_B = "01" else
    RS_wb; -- Forwarded value for source 2 operand

source_1 <= a_fw when source_1_select = '0' else
  PC_in when source_1_select = '1' else
    (others => '0'); -- Select source 1 operand

source_2 <= b_fw when source_2_select = '0' else
  IMM_in when source_2_select = '1' else
    (others => '0'); -- Select source 2 operand

alu_out <= result; -- Assert ALU output
EXECUTE_PROCESS : process (clk, reset)
begin
  if reset = '1' then
    result <= (others => '0'); -- Reset result to zero
    MEM_data_out <= (others => '0'); -- Reset memory data output to zero
    RD_out <= (others => '0'); -- Reset register file data output to zero

  elsif rising_edge(clk) then
    if stage_enable = '1' then
      if alu_op = RTYPE_ADD then
        result <= std_logic_vector(signed(source_1) + signed(source_2)); -- ADD
          ↳ operation
      elsif alu_op = RTYPE_SUB then
        result <= std_logic_vector(signed(source_1) - signed(source_2)); -- SUB
          ↳ operation
      elsif alu_op = RTYPE_AND then
        result <= source_1 and source_2; -- AND operation
      elsif alu_op = RTYPE_OR then
        result <= source_1 or source_2; -- OR operation
      elsif alu_op = RTYPE_XOR then
        result <= source_1 xor source_2; -- XOR operation
      elsif alu_op = RTYPE_SLL then
        result <= std_logic_vector(shift_left(unsigned(source_1),
          ↳ to_integer(unsigned(source_2)))); -- SLL (Shift Left Logical)
      elsif alu_op = RTYPE_SRL then
        result <= std_logic_vector(shift_right(unsigned(source_1),
          ↳ to_integer(unsigned(source_2)))); -- SRL (Shift Right Logical)
      elsif alu_op = RTYPE_SNE then -- SNE (Set Not Equal)
        if unsigned(source_1) /= unsigned(source_2) then
          result <= std_logic_vector(to_signed(1, result'length));
        else

```

```

        result <= (others => '0');
    end if;
elsif alu_op = RTYPE_SLE then -- SLE (Set Less Than or Equal)
    if unsigned(source_1) <= unsigned(source_2) then
        result <= std_logic_vector(to_signed(1, result'length));
    else
        result <= (others => '0');
    end if;
elsif alu_op = RTYPE_SGE then -- SGE (Set Greater Than or Equal)
    if unsigned(source_1) >= unsigned(source_2) then
        result <= std_logic_vector(to_signed(1, result'length));
    else
        result <= (others => '0');
    end if;
else
    result <= (others => '0'); -- Default case, reset result
end if;

-- case alu_op_int is
--   when RTYPE_ADD => -- ADD
--       result <= std_logic_vector(unsigned(source_1) + unsigned(source_2));
--   when RTYPE_SUB => -- SUB
--       result <= std_logic_vector(unsigned(source_1) - unsigned(source_2));
--   when RTYPE_AND => -- AND
--       result <= source_1 and source_2;
--   when RTYPE_OR => -- OR
--       result <= source_1 or source_2;
--   when RTYPE_XOR => -- XOR
--       result <= source_1 xor source_2;
--   when RTYPE_SLL => -- SLL (Shift Left Logical)
--       result <= std_logic_vector(shift_left(unsigned(source_1),
-- → to_integer(unsigned(source_2))));
--   when RTYPE_SRL => -- SRL (Shift Right Logical)
--       result <= std_logic_vector(shift_right(unsigned(source_1),
-- → to_integer(unsigned(source_2))));
--   when RTYPE_SNE => -- SNE (Set Not Equal)
--       if unsigned(source_1) /= unsigned(source_2) then
--           result <= std_logic_vector(to_signed(1, result'length));
--       else
--           result <= (others => '0');
--       end if;
--   when RTYPE_SLE => -- SLE (Set Less Than)
--       if unsigned(source_1) <= unsigned(source_2) then
--           result <= std_logic_vector(to_signed(1,
-- → result'length));
--       else
--           result <= (others => '0');
--       end if;
--   when RTYPE_SGE => -- SGE (Set Greater Than)
--       if unsigned(source_1) >= unsigned(source_2) then

```

```

--      result <= std_logic_vector(to_signed(1, result'length));
--    else
--      result <= (others => '0');
--    end if;
--    when others      =>
--      result <= (others => '0'); -- Default case, reset result
--    end case;
RD_out      <= RD_in; -- Assert register file write address for destination
--    register
MEM_data_out <= b_fw; -- Assert memory data output (for memory operations)
end if;
end if;
end process;

end Behavioral;
=====

```

A.6 Memory and Write Back Stage

```

use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.math_real.all;
=====
-- Entity Declaration
entity write_mem_rf_stage is
  generic (
    OPERAND_SIZE : integer := 32; -- Size operands in bits
    NUM_REG      : integer := 5 -- Number of registers in bits (log2(RF_SIZE))
  );
  port (
    -- DLX Processor signals
    clk      : in std_logic; -- Clock signal
    reset    : in std_logic; -- Reset signal
    -- Control unit signals
    stage_enable : in std_logic; -- stage enable
    DRAM_enable  : in std_logic; -- DRAM enable signal
    DRAM_RE      : in std_logic; -- DRAM read enable signal
    DRAM_WE      : in std_logic; -- DRAM write enable signal
    source_select : in std_logic; -- Source select signal MEM/ALU
    -- Input/Output signals
    alu_result    : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- ALU result
    --    input
    MEM_data_in   : in std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory data
    --    input (from execute stage)
    MEM_stage_out : out std_logic_vector(OPERAND_SIZE - 1 downto 0); -- Memory stage
    --    output

    -- Register file signals

```

```

    RD_in  : in std_logic_vector(NUM_REG - 1 downto 0); -- Register file write
    ↪ address for destination register input
    RD_out : out std_logic_vector(NUM_REG - 1 downto 0) -- Register file address
    ↪ input (WB stage)
);
end write_mem_rf_stage;
=====
architecture Behavioral of write_mem_rf_stage is
    component DRAM is
        generic (
            DATA_SIZE : integer := 32; -- Size of instruction in bits
            RAM_DEPTH  : integer := 48 -- DRAM Size
        );
        port (
            Rst : in std_logic;
            re  : in std_logic; -- Read enable signal
            we  : in std_logic; -- Write enable signal
            -- Address and data ports
            Addr      : in std_logic_vector(DATA_SIZE - 1 downto 0);
            data_in   : in std_logic_vector(DATA_SIZE - 1 downto 0);
            data_out  : out std_logic_vector(DATA_SIZE - 1 downto 0)
        );
    end component DRAM;

    signal DRAM_out : std_logic_vector(OPERAND_SIZE - 1 downto 0);

begin

    DRAM_inst : DRAM
    generic map(
        DATA_SIZE => OPERAND_SIZE, -- Size of data in bits
        RAM_DEPTH  => 48 -- DRAM Size (increase if needed)
    )
    port map
    (
        rst      => reset,
        re       => DRAM_RE,
        we       => DRAM_WE,
        Addr     => alu_result,
        data_in  => MEM_data_in,
        data_out => DRAM_out
    );

    MEM_STAGE : process (clk, reset)
    begin
        if reset = '1' then

            MEM_stage_out <= (others => '0'); -- Reset memory stage output

```

```

RD_out      <= (others => '0'); -- Reset register file data output

elsif rising_edge(clk) then

    if stage_enable = '1' then
        RD_out <= RD_in;
        -- Output of memory stage
        if source_select = '0' then
            MEM_stage_out <= alu_result; -- Forward ALU result to Write back stage
        else
            MEM_stage_out <= DRAM_out; -- Forward DRAM output to Write back stage
        end if;
    end if;

end if;
end process MEM_STAGE;
end Behavioral;
=====

```

A.7 Forwarding Unit

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use std.textio.all;
use ieee.math_real.all;
use work.myTypes.all;
use ieee.std_logic_textio.all;
=====
-- Entity Declaration
entity forwarding_unit is
    generic (
        OPERAND_SIZE : integer := 32; -- Size operands in bits
        NUM_REG       : integer := 5 -- Number of registers in bits (log2(RF_SIZE))
    );
    port (
        -- Control unit signals
        stage_enable : in std_logic; -- Stage enable signal

        -- Input/Output signals
        EX_MEM_RD    : in std_logic_vector(NUM_REG - 1 downto 0); -- Destination register
        --   ↪ from EX stage
        MEM_WB_RD     : in std_logic_vector(NUM_REG - 1 downto 0); -- Destination register
        --   ↪ from MEM stage
        EX_MEM_RegW   : in std_logic; -- Register write enable from MEM stage
        MEM_WB_RegW   : in std_logic; -- Register write enable from WB stage
        ID_RS_1       : in std_logic_vector(NUM_REG - 1 downto 0); -- Source register 1
        --   ↪ from EX stage
    );
end forwarding_unit;

```

```

ID_RS_2      : in std_logic_vector(NUM_REG - 1 downto 0); -- Source register 2
↳ from EX stage
opcode       : in std_logic_vector(5 downto 0); -- Opcode of the instruction
fwd_A        : out std_logic_vector(1 downto 0); -- Forwarding control for source
↳ A
fwd_B        : out std_logic_vector(1 downto 0); -- Forwarding control for source
↳ B
stall        : out std_logic -- Stall signal to control data hazards
);
end forwarding_unit;

-- Additional comments:
-- EX_MEM_RegW MEM_WB_RegW are used to catch if the result will be written back to
↳ the register file
=====
architecture Behavioral of forwarding_unit is

begin

fwd_A <= "01" when (EX_MEM_RegW = '1' and ID_RS_1 = EX_MEM_RD) else -- Forward
↳ from EX stage if the destination register matches source A
  "11" when (MEM_WB_RegW = '1' and ID_RS_1 = MEM_WB_RD) else -- Forward from MEM
↳ stage if the destination register matches source A
  (others => '0'); -- Default no forwarding for source A

fwd_B <= "01" when (EX_MEM_RegW = '1' and ID_RS_2 = EX_MEM_RD) else -- Forward
↳ from EX stage if the destination register matches source B
  "11" when (MEM_WB_RegW = '1' and ID_RS_2 = MEM_WB_RD) else -- Forward from MEM
↳ stage if the destination register matches source B
  (others => '0'); -- Default no forwarding for source B

-- Stall if the operation is a load and the source registers match the destination
↳ register of the EX stage
stall <= '0' when (opcode = ITYPE_LW and (ID_RS_1 = EX_MEM_RD or ID_RS_2 =
↳ EX_MEM_RD)) else
  '1'; -- Stall if the load instruction is in progress and source registers match
↳ the destination register of the EX stage

end Behavioral;
=====

```

APPENDIX B

User Scripts

B.1 Pupi

```
package main

import (
    "bufio"
    "fmt"
    "io"
    "os"
    "slices"
    "strings"
)

// the basics is that every file that needs a component
// is put in a waiting line until all it's requests are
// satisfied

type node struct {
    Name      string
    Got, Want map[string]struct{}
}

func main() {
    content, err := io.ReadAll(os.Stdin)
    if err != nil {
        panic(err)
    }
    files := strings.Split(string(content), "\n")
    nodes := make([]*node, 0, len(files))
    for _, name := range files {
        if strings.TrimSpace(name) == "" {
            continue
        }
    }
}
```

```

nn := &node{
    Name: name,
    Got:  map[string]struct{}{},
    Want: map[string]struct{}{},
}

f, err := os.Open(name)
if err != nil {
    panic(err)
}

s := bufio.NewScanner(f)

// need to add priority to the packages
// Types of files
// Package only with constants => no entities found first to compile
//   ↳ (No Got no Want => can bbe directly compiled)
// Package with entities => need to sweep for entites and functions
//   ↳ (Surely has Got and can have Want)
// Entity => if no components needed has only Got but can request
//   ↳ some packages for constants
// Top Entity => can request other components "entities" has Want
//   ↳ and less got

// we can sweep if package or not, in general:
// we see what entities declaration has and we put in the got
// we see what components needs and put in the want
// extra: we see what functions declares and put in the got
// extra: we see what functions needs and put in the want
// if no Want and no Got we output it (is a package with only
//   ↳ constants)

for s.Scan() {
    line := s.Text()
    line = strings.ToLower(line)

    // we need only the declatarions
    if strings.Contains(line, ";") {
        continue
    }

    ff := strings.Fields(line)

    if len(ff) == 0 {
        continue
    }

    // the component keyword means a request => thus a Want
    if idx := slices.Index(ff, "component"); idx != -1 {

```

```

        name := ff[idx+1]
        nn.Want[name] = struct{}{}
        continue
    }

    // the entity keyword means an entity declaration => thus a
    ↪ got
    if idx := slices.Index(ff, "entity"); idx != -1 {
        name := ff[idx+1]
        nn.Got[name] = struct{}{}
        continue
    }

    //If nothing found it will have Want and Got empty
}
nodes = append(nodes, nn)
}

//first we remove the ones without requests
todel := []*node{}

for _, nn := range nodes {
    // Simple package type
    if nn != nil {
        if len(nn.Want) == 0 && len(nn.Got) == 0 {
            // compile it
            fmt.Println(nn.Name)
            todel = append(todel, nn)
            // Delete the actual compiled node
            nodes = slices.DeleteFunc(nodes, func(n *node) bool
                ↪ { return slices.Contains(todel, n) })
        }
    }
}

// We iterate ass long as there is a file with a want
// at each iteration we compile the nodes with only got
// instances
for {
    if len(nodes) == 0 {
        break
    }
    todel := []*node{}

    for _, nn := range nodes {
        // Exclude the top level entity
        if len(nn.Want) != 0 {
            continue
        }
    }
}

```

```

        // We only give out things so we compile and erase from the
        // consumers
        if len(nn.Want) == 0 {
            fmt.Println(nn.Name)
        }

        todel = append(todel, nn)
        // delete from the other nodes what i just have compiled
        for _, consumer := range nodes {
            for what := range nn.Got {
                delete(consumer.Want, what)
            }
        }
    }
    // Delete the actual compiled node
    nodes = slices.DeleteFunc(nodes, func(n *node) bool { return
        ↪ slices.Contains(todel, n) })
}
}

```

B.2 Marley

```

#!/bin/sh

## Utils scripts
source ./scripts/utils/script_utils.sh
## Main functions scripts
source ./scripts/briciola.sh
source ./scripts/scorzetta.sh
source ./scripts/boh.sh
source ./scripts/cassiopeia.sh

# Main script - marley with continuous loop
clear
e_header "Welcome I'm Marley"
e_note "Using a terminal with color support is raccomanded!"
e_bold "I'm the Microelectronics laboratory helper"
while true; do
    # Clear the screen (optional)
    # Display menu options
    e_bold "What do you want to do?"
    e_header "Menu"
    echo "1: Setup lab folder"
    echo "2: Run Briciola    (Compilation)"
    echo "3: Run Scorzetta    (Simulation)"
    echo "4: Run Boh           (Synthesis)"
    echo "5: Run Bohx2          (Phisical Design)"

```

```

echo "6: Exit"
e_header "===="

# Read user input
read -p "Enter your choice (1-4): " COMMAND

# Process user input
case $COMMAND in
1)
    if [ ! -d ./exclude ]; then
        mkdir ./exclude
    fi
    if [ ! -d ./work ]; then
        mkdir ./work
    fi
    # TODO: Copy the scripts from the user home folder
    # TODO: Copy the component folder
    ;;
2)
    # Compilation
    echo "Starting Briciola..."
    briciola
    read -p "Press [Enter] to return to Marley menu..."
    ;;
3)
    # Simulation
    # compile the asm file
    e_bold "Enter the ASM file to compile (taken from ./asm/ folder !!!only
↳ name!!! extension will automatically added):"
    read ASM_FILE
    if [ ! -f "./asm/${ASM_FILE}.asm" ]; then
        e_error "ASM file not found in ./asm/ folder."
        read -p "Press [Enter] to return to Marley menu..."
        continue
    fi

    e_arrow "Compiling $ASM_FILE..."
    rm ./test.asm.mem
    perl ./scripts/dlxasm.pl ./asm/${ASM_FILE}.asm && ./scripts/conv2memory
↳ ./asm/${ASM_FILE}.asm.exe >test.asm.mem

    if [ ! -f ./test.asm.mem ]; then
        e_error "Memory file not created - ASM compilation failed, please check
↳ the ASM file."
        read -p "Press [Enter] to return to Marley menu..."
        continue
    fi
    # Remove unneeded files
    rm ./asm/${ASM_FILE}.asm.exe

```

```

rm ./asm/${ASM_FILE}.asm.exe.hdr

# Start Scorzetta for the simulation
if [ -d ./synthesys ]; then
    e_warning "It seems that this is a Simualtion folder!"
    while true; do
        read -r -p "Do you want to continue? (y/n)" ANSWER
        case $ANSWER in
            [Yy]*)
                echo "Starting Scorzetta..."
                scorzetta
                break
                ;;
            [Nn]*) break ;;
            *) e_error "Please answer Y or N." ;;
        esac
    done
else
    echo "Starting Scorzetta..."
    scorzetta
fi

read -p "Press [Enter] to return to Marley menu..."
;;

4)

# Synthesis
if [ -d ./simulation ]; then
    e_warning "It seems that this is a Simualtion folder!"
    while true; do
        read -r -p "Do you want to continue? (y/n)" ANSWER
        case $ANSWER in
            [Yy]*)
                echo "Starting boh..."
                boh
                break
                ;;
            [Nn]*) break ;;
            *) e_error "Please answer Y or N." ;;
        esac
    done
else
    echo "Starting boh..."
    boh
fi

read -p "Press [Enter] to return to Marley menu..."
;;

5)

# Phisical Design

```

```

# if no previous synthesis is present give error
if [ ! -d ./synthesys ]; then
    e_warning "It seems that there was no previous synthesis done by Boh"
    while true; do
        read -r -p "Do you want to continue? (y/n)" ANSWER
        case $ANSWER in
            [Yy]*) break ;;
            [Nn]*) exit ;;
            *) e_error "Please answer Y or N." ;;
        esac
    done
fi

echo "Starting Cassiopeia..."
cassiopeia
read -p "Press [Enter] to return to Marley menu..."
;;

6)
    e_header "Goodbye!"
    exit 0
    ;;

*)
    echo "Invalid choice. Please enter 1,2,3,4,5."
    read -p "Press [Enter] to try again..."
    ;;
esac
clear
done

```

B.3 Briciola

```

#!/bin/sh

function briciola() {
    orange=$(tput setaf 69)

    e_header "    Briciola    "
    cat ./scripts/utills/cat2.txt
    e_header "Compilation tool"

    # Create the hiearachy
    find -type f -name "*.vhd" -not -path "./work/*" -not -path "./exclude/*" -not
    ↪ -path "./simulations/*" | tr " " "\n" | ./scripts/utills/pupi > compile.list
    if [ $? -ne 0 ]; then
        e_error "Error in creating compile.list"
        exit 1
    fi

    e_success "Created compile.list file"
}

```

```

# Append files in the compile.list file
# and remove trailing spaces or empty lines
while IFS="" read -r p || [ -n "$p" ]
do
    VHDL_FILES+=($(echo "$p" | xargs))
done < compile.list

# Compile the Hierarchy
# if /eda/scripts/init_questa_core_prime exists, source it
if [ -f /eda/scripts/init_questa_core_prime ]; then
    source /eda/scripts/init_questa_core_prime
fi

if [ ! -e work/ ]; then
    mkdir -p work/
fi

vlib work

for i in "${VHDL_FILES[@]}"
do
    e_header "Started Compilation of file: $i"
    vcom -work work "$i"
    if [ $? -ne 0 ]; then
        e_error "Compilation of file $i [ FAILED ]"
        return
        exit 1
    else
        e_success "Compilation of file $i [ OK ]"
    fi
    e_header " Ended Compilation of file: $i "
done

e_header " Exiting Briciola "
e_header " Compilation Ended "

orange=$(tput setaf 215)
}

```

B.4 Scorzetta

```

#!/bin/bash

function scorzetta() {
    ## UI things
    orange=$(tput setaf 215)

    VHDL_FILES=()
    WORK_DIR=$(pwd)

```

```

e_header "    Scorzetta    "
cat ./scripts/utills/cat1.txt
e_header "Simulation tool"

e_note "Using a terminal with color support is raccomended!"

## Check and ask if the user wants to run briciola before simulating
if [ ! -e compile.list ]; then
    e_warning "It seems that there was no previous compilation done by Briciola"
    while true; do
        read -r -p "Do you want to continue? (y/n)" ANSWER
        case $ANSWER in
            [Yy]*) break ;;
            [Nn]*) exit ;;
            *) e_error "Please answer Y or N." ;;
        esac
    done
fi

while true; do
    read -r -p "Do you want to run Briciola to compile the files? (y/n)" ANSWER
    case $ANSWER in
        [Yy]*)
            briciola
            break
            ;;
        [Nn]*) break ;;
        *) e_error "Please answer Y or N." ;;
    esac
done

# If mode simulation are needed ask to name ad put the output files in
# a separate folder
while true; do
    read -r -p "Do you want to give a name to the current simulation? (y/n)"
    ↪ ANSWER
    case $ANSWER in
        [Yy]*)
            read -r -p "Simualation name: " SIM_NAME
            WORK_DIR="$WORK_DIR/simulations/$SIM_NAME"
            echo $WORK_DIR

            if [ ! -d "$WORK_DIR" ]; then
                mkdir -p $WORK_DIR
            else
                e_warning "Simulation with the same name present."
                while true; do
                    read -r -p "Do you want to overwrite it? (y/n)" ANSWER

```

```

        case $ANSWER in
            [Yy]*)
                rm $WORK_DIR/*
                break
                ;;
            [Nn]*)
                e_warning "Exiting Scorzetta"
                exit
                ;;
            *) e_error "Please answer Y or N." ;;
        esac
    done
fi
break
;;
[Nn]*) break ;;
*) e_error "Please answer Y or N." ;;
esac
done
if [ -f /eda/scripts/init_questa_core_prime ]; then
    source /eda/scripts/init_questa_core_prime
fi

## TODO: get the available entities or configurations from the library
# *****
# * Entity: TB_ALU          *
# * Available Configurations: *
# *   TEST_BEH_ALU         *
# *   TEST_STR_ALU         *
# * Entity: ALU            *
# * Available Configurations: *
# *   STR_ALU              *
# *   BEH_ALU              *
# *****

## Ask for simulation parameters
e_bold "Enter simulation resolution: Ex. "1ns""
read SIM_RES
e_bold "Enter the simulation time: Ex "40ns""
read SIM_TIME
e_bold "Enter the name of the top level entity to be simulated:"
read TOP_LEVEL_ENTITY

WAVEFORM_FILE="$WORK_DIR/waveform.vcd"
MODELSIM_FILE="$WORK_DIR/waveform.wlf"

## TODO: add custom simulation file
# Build the tcl file with the user defined variables
## NOT SAFE WE ARE USING EVAL, but we don't need to export variables

```

```

TCL_FILE=$(eval "echo
↪ \"$(<./scripts/scripts_templates/simulation_template.tcl)\" 2>/dev/null)

## Safer but we need to export the variables
# export TOP_LEVEL_ENTITY SIM_RES SIM_TIME WAVEFORM_FILEe
# TCL_FILE=$(cat ./scripts_templates/simulation_template.tcl | envsubst)

# Run the tcl template
# If you must call lfinish for some reason (e.g. it's called from within your
↪ test framework, and you can't modify it), you can get your exit code out to
↪ the caller using "onfinish stop". Example:

# vsim -c test_bench -do "onfinish stop; run -all"
vsim -c -do "$TCL_FILE"

# Check vsim eexit status
if [ $? -ne 0 ]; then
    e_error "Simulation of $TOP_LEVEL_ENTITY    [ FAILED ]"
    exit 1
else
    e_success "Simulation of $TOP_LEVEL_ENTITY    [ OK ]"
fi

# If the waveform file is created
# Create also a waveform file for modelsim, so we can use
# both gtkwave and modelsim to view it
if [ ! -e "$WAVEFORM_FILE" ]; then
    e_success "Creation of the waveform file    [ OK ]"
    e_arrow "Creating WLF modelsim waveform file..."
    vcd2wlf $WAVEFORM_FILE $MODELSIM_FILE
    if [ $? -ne 0 ]; then
        e_error "Creating WLF modelsim waveform file    [ FAILED ]"
        exit 1
    else
        e_success "Creating WLF modelsim waveform file    [ OK ]"
        e_note "Simulation finished, results are in $WORK_DIR"
        e_note "You can find the waveform file in $WORK_DIR/waveform.vcd"
        e_note "You can find the modelsim waveform file in
↪ $WORK_DIR/waveform.wlf"
        while true; do
            e_arrow "Do you want to open the waveform file with gtkwave(Gg) or
↪ modelsim(Mm) or exit(Nn)?"
            read -r -p "Choice (G/g, M/m or N/n): " WAVEFORM_VIEWER
            case $WAVEFORM_VIEWER in
                [Gg]*)
                    if ! type gtkwave >/dev/null; then
                        # install foobar here
                        e_error "gtkwave not found"
                    fi
                ;;
            esac
        done
    fi
fi

```

```

        continue
    fi

    gtkwave "$WAVEFORM_FILE" &
    >/dev/null
    break
    ;;
[Mm]*)
    if ! type vsim >/dev/null; then
        # install foobar here
        e_error "ModelSim/QuuestaSim not found"
        continue
    fi
    vsim -view "$MODELSIM_FILE" &
    >/dev/null
    break
    ;;
[Nn]*)
    e_note "Oke exiting Scorzetta"
    break
    ;;
*)
    e_error "Please answer G/g, M/m or N/n."
    ;;
esac
done
fi
else
    e_error "Creation of the waveform file    [ FAILED ]"
fi

# TODO: we can maybe integrate pushover to tell when the sim/synt is finished
## Revert UI things
orange=$(tput setaf 215)
}

```

B.5 Boh

```

#!/bin/bash

function boh() {
    ## UI things
    orange=$(tput setaf 215)

    VHDL_FILES=()
    WORK_DIR=$(pwd)

    e_header "    boh    "
    cat ./scripts/utils/boh.txt
}

```

```

e_header "Synthesis tool"

## Check and ask if the user wants to run briciola before simulating
if [ ! -e compile.list ]; then
    e_warning "It seems that there was no previous compilation done by Briciola"
    while true; do
        read -r -p "Do you want to continue? (y/n)" ANSWER
        case $ANSWER in
            [Yy]*) break ;;
            [Nn]*) exit ;;
            *) e_error "Please answer Y or N." ;;
        esac
    done
fi
while true; do
    read -r -p "Do you want to run Briciola to compile the files? (y/n)" ANSWER
    case $ANSWER in
        [Yy]*)
            briciola
            break
            ;;
        [Nn]*) break ;;
        *) e_error "Please answer Y or N." ;;
    esac
done

# If mode synthesis are needed ask to name ad put the output files in
# a separate folder
while true; do
    read -r -p "Do you want to give a name to the synthesis? (y/n)" ANSWER
    case $ANSWER in
        [Yy]*)
            read -r -p "Synthesis name: " SYN_NAME
            WORK_DIR="$WORK_DIR/synthesis/$SYN_NAME"
            if [ ! -d $WORK_DIR ]; then
                mkdir -p $WORK_DIR
                mkdir -p $WORK_DIR/timing
                mkdir -p $WORK_DIR/power
                mkdir -p $WORK_DIR/area
            else
                echo "Synthesis with the same name present."
                while true; do
                    read -r -p "Do you want to overwrite it? (y/n)" ANSWER
                    case $ANSWER in
                        [Yy]*)
                            rm $WORK_DIR/*
                            mkdir -p $WORK_DIR/timing
                            mkdir -p $WORK_DIR/power
                    esac
                done
            fi
        esac
    done
done

```

```

        mkdir -p $WORK_DIR/area
        mkdir -p $WORK_DIR/clock
        mkdir -p $WORK_DIR/synthesized_constraints
        mkdir -p $WORK_DIR/structural
        break
    ;;
[Nn]*)
    e_warning "Exiting Boh"
    exit
    ;;
*) e_error "Please answer Y or N." ;;
esac
done
fi
break
;;
[Nn]*) break ;;
*) e_error "Please answer Y or N." ;;
esac
done

## TODO: get the available entities or configurations from the library
# *****
# * Entity: TB_ALU *
# * Available Configurations: *
# *   TEST_BEH_ALU *
# *   TEST_STR_ALU *
# * Entity: ALU *
# * Available Configurations: *
# *   STR_ALU *
# *   BEH_ALU *
# *****

## Ask for simulation parameters
echo "Enter the name of the top level entity to be Synthesized:"
read TOP_LEVEL_ENTITY
echo "Enter the the name of the architecture/configuration of $TOP_LEVEL_ENTITY
↳ to be Synthesized:"
read ENTITY_CONF

## Design Vision setup
cp /home/repository/ms/setup/.synopsys_dc.setup .
source /eda/scripts/init_design_vision

e_header "Starting Synthesis"
# Append files in the compile.list file
# and remove trailing spaces or empty lines
VHDL_FILES=$(cat ./compile.list)

```

```

## TODO: add custom synthesis file
# Build the tcl file with the user defined variables
## NOT SAFE WE ARE USING EVAL, but we don't need to export variables

TCL_FILE=$(eval "echo
→ \"$(<./scripts/scripts_templates/synthesis_template.tcl)\" 2>/dev/null)
echo \"$TCL_FILE\" >./scripts/tmp.tcl

## Safer but we need to export the variables
# export TOP_LEVEL_ENTITY SIM_RES SIM_TIME WAVEFORM_FILEe
# TCL_FILE=$(cat ./scripts_templates/synthesis_template.tcl | envsubst)

# Run the tcl template
dc_shell-xg-t -f ./scripts/tmp.tcl
# vsim -c -do "$TCL_FILE"

# Check vsim eexit status
if [ $? -ne 0 ]; then
    e_error "Synthesis of $TOP_LEVEL_ENTITY    [ FAILED ]"
    exit 1
else
    e_success "Synthesis of $TOP_LEVEL_ENTITY    [ OK ]"
fi
rm ./scripts/tmp.tcl
# If the waveform file is created
# Create also a waveform file for modelsim, so we can use
# both gtkwave and modelsim to view it
# if [ -e $WORK_DIR/report_area.txt ]; then
#     e_success "Creation of the report_area file    [ OK ]"
# else
#     e_error "Creation of the report_area file    [ FAILED ]"
# fi
# if [ -e $WORK_DIR/report_timing.txt ]; then
#     e_success "Creation of the report_area file    [ OK ]"
# else
#     e_error "Creation of the report_area file    [ FAILED ]"
# fi
# if [ -e $WORK_DIR/report_power.txt ]; then
#     e_success "Creation of the report_area file    [ OK ]"
# else
#     e_error "Creation of the report_area file    [ FAILED ]"
# fi

# TODO: we can maybe integrate pushover to tell when the sim/synt is finished

## Revert UI things
orange=$(tput setaf 215)
}

```

B.6 Cassiopeia

```
#!/bin/sh

function cassiopeia() {
    e_header "    Cassiopeia    "
    cat ./scripts/utils/stars.txt
    e_header "Physical Design tool"
    # Cassiopeia script to run innouvs with specified parameters
    # clean previous runs
    if [ -d ./physical_design ]; then
        rm -rf ./physical_design/*
    fi else
        mkdir -p ./physical_design
    fi

    touch ./physical_design/physical_design.log

    read -r -p "Enter the name of the synthesis iteration of what you want to do
    ↪ physical design" SYN_FOLDER
    if [ ! -d "./synthesis/$SYN_FOLDER" ]; then
        e_error "Synthesis folder $SYN_FOLDER does not exist."
        exit 1
    fi

    read -r -p "Enter the name of the top level synthetized " TOP_LEVEL
    if [ ! -f "./synthesis/structural/${TOP_LEVEL_ENTITY}-structural-no-opt.v" ];
    ↪ then
        e_error "Structural files for $TOP_LEVEL does not exist."
        exit 1
    fi

    mkdir -p ./physical_design/designs
    mkdir -p ./physical_design/constraints
    mkdir -p ./physical_design/timingReport
    cp ./synthesis/$SYN_FOLDER/structural/*.v ./physical_design/designs/
    cp ./synthesis/$SYN_FOLDER/synthesized_constraints/*.sdc
    ↪ ./physical_design/constraints/

    if [ -f /eda/scripts/init_cadence_2020-21 ]; then
        source /eda/scripts/init_cadence_2020-21
    fi

    # Check if the physical design script template exists
    if [ ! -f ./scripts/scripts_templates/physical_design.tcl ]; then
        e_error "Physical design script template not found."
        exit 1
    fi
}
```

```
e_header "Starting Physical Design with Innouvs"
e_note "This may take a while, please be patient."
# Run the innouvs command with the physical design script
innouvs -no_gui -files ./scripts/scripts_templates/physical_design.tcl -log
↳ ./physical_design/physical_design.log
```

```
}
```

APPENDIX C

TCL templates

C.1 Simualtion

```
# Available variables from the bash script
# ℓ{TOP_LEVEL_ENTITY} : Name of the top level entity to simulate
# ℓ{SIM_TIME} : Simualtion Time
# ℓ{SIM_RES} : Simualtion resolution
# ℓ{WAVEFORM_FILE} : Output waveform file
# PS: if you need more info from the user given to
# the bash script you obv need to modify the
# bash script to export/eval those variables

# Load the simulation
# TODO: maybe ask for additional arguments?
vsim -t ${SIM_RES} -voptargs+acc ${TOP_LEVEL_ENTITY}

# Open a waveform file to dump the simulaiton
vcd file ${WAVEFORM_FILE}
vcd add -r *
# add wave /*

# Run the simulation for the specified time
run ${SIM_TIME}

# wave zoom full
# Save the waveform file
# write wave waveform.ps -landscape -perpage 6ns
# view structure
# Exit Modelsim
quit;
```

C.2 Synthesys

```
# Available variables from the bash script
#   WORK_DIR: Directory where the synthesis results will be stored
#   subfolders:
#       - /timing: timing reports
#       - /area: area reports
#       - /power: power reports
#       - /clk: clock reports
#       - /synthesized_constraints: synthesized constraints
#       - /structural: structural VHDL files
#   VHDL_FILES: List of VHDL files to be synthesized
#   TOP_LEVEL_ENTITY: Name of the top-level entity to be synthesized
#   ENTITY_CONF: Configuration of the top-level entity (e.g., architecture name)
# PS: if you need more info from the user given to
# the bash script you obv need to modify the
# bash script to export/eval those variables

# Set the libraries
set library WORK
define_design_lib WORK -path work
analyze -library WORK -format vhd1 ${VHDL_FILES}
elaborate ${TOP_LEVEL_ENTITY} -architecture ${ENTITY_CONF} -library work

# Without constraints
compile -map_effort high
# compile_ultra -timing_high_effort_script

# reporting timing and area after the first synthesis without constraints #
report_timing -nworst 10 >
↪ ${WORK_DIR}/timing/${TOP_LEVEL_ENTITY}-report-timing-no-opt.txt
report_area > ${WORK_DIR}/area/${TOP_LEVEL_ENTITY}-report-area-no-opt.txt
report_clock > ${WORK_DIR}/clk/${TOP_LEVEL_ENTITY}-report-clk-no-opt.txt

write -hierarchy -format verilog -output
↪ ${WORK_DIR}/structural/${TOP_LEVEL_ENTITY}-structural-no-opt.v

write_sdc
↪ ${WORK_DIR}/synthesized_constraints/${TOP_LEVEL_ENTITY}-synthesized_constraints-no-opt.sdc

## Only with clock constraint
create_clock -name "clk" -period 2 clk
compile -map_effort high
# compile_ultra -timing_high_effort_script

report_timing -nworst 10 >
↪ ${WORK_DIR}/timing/${TOP_LEVEL_ENTITY}-report-timing-clkopt.txt
report_area > ${WORK_DIR}/area/${TOP_LEVEL_ENTITY}-report-area-clkopt.txt
report_clock > ${WORK_DIR}/clk/${TOP_LEVEL_ENTITY}-report-clk-clkopt.txt
```

```
write -hierarchy -format verilog -output
↳ ${WORK_DIR}/structural/${TOP_LEVEL_ENTITY}-structural-clkopt.v

write_sdc
↳ ${WORK_DIR}/synthesized_constraints/${TOP_LEVEL_ENTITY}-synthesized_constraints-clkopt.sdc

## With both type of constraints
create_clock -name "clk" -period 2 clk
set_max_delay 2 -from [all_inputs] -to [all_outputs]
compile -map_effort high
# compile_ultra -timing_high_effort_script

report_timing -nworst 10 >
↳ ${WORK_DIR}/timing/${TOP_LEVEL_ENTITY}-report-timing-totopt.txt
report_area > ${WORK_DIR}/area/${TOP_LEVEL_ENTITY}-report-area-totopt.txt
report_clock > ${WORK_DIR}/clk/${TOP_LEVEL_ENTITY}-report-clk-totopt.txt

write -hierarchy -format verilog -output
↳ ${WORK_DIR}/structural/${TOP_LEVEL_ENTITY}-structural-totopt.v
write_sdc
↳ ${WORK_DIR}/synthesized_constraints/${TOP_LEVEL_ENTITY}-synthesized_constraints-totopt.sdc

## With strict constraints
create_clock -name "clk" -period 1 clk
set_max_delay 0.5 -from [all_inputs] -to [all_outputs]
compile -map_effort high
# compile_ultra -timing_high_effort_script

report_timing -nworst 10 >
↳ ${WORK_DIR}/timing/${TOP_LEVEL_ENTITY}-report-timing-strictopt.txt
report_area > ${WORK_DIR}/area/${TOP_LEVEL_ENTITY}-report-area-strictopt.txt
report_clock > ${WORK_DIR}/clk/${TOP_LEVEL_ENTITY}-report-clk-strictopt.txt

write -hierarchy -format verilog -output
↳ ${WORK_DIR}/structural/${TOP_LEVEL_ENTITY}-structural-strictopt.v
write_sdc
↳ ${WORK_DIR}/synthesized_constraints/${TOP_LEVEL_ENTITY}-synthesized_constraints-strictopt.sdc

# Exit
exit
```

C.3 Physical Design

C.3.1 Default.view

```
# Version:1.0 MMMC View Definition File
# Do Not Remove Above Line
create_rc_corner -name standard -cap_table $MycapTable -T {300} -preRoute_res {1.0}
↳ -preRoute_cap {1.0} -preRoute_clkres {0.0} -preRoute_clkcap {0.0} -postRoute_res
↳ {1.0} -postRoute_cap {1.0} -postRoute_xcap {1.0} -postRoute_clkres {0.0}
↳ -postRoute_clkcap {0.0}
create_rc_corner -name high -cap_table $MycapTable -T {400} -preRoute_res {1.0}
↳ -preRoute_cap {1.0} -preRoute_clkres {0.0} -preRoute_clkcap {0.0} -postRoute_res
↳ {1.0} -postRoute_cap {1.0} -postRoute_xcap {1.0} -postRoute_clkres {0.0}
↳ -postRoute_clkcap {0.0}
create_rc_corner -name low -cap_table $MycapTable -T {240} -preRoute_res {1.0}
↳ -preRoute_cap {1.0} -preRoute_clkres {0.0} -preRoute_clkcap {0.0} -postRoute_res
↳ {1.0} -postRoute_cap {1.0} -postRoute_xcap {1.0} -postRoute_clkres {0.0}
↳ -postRoute_clkcap {0.0}
create_library_set -name libsTYP -timing $MyTimingLibNom
create_library_set -name libsWC -timing $MyTimingLibSlow
create_library_set -name libsBC -timing $MyTimingLibFast
# Load the design constraints
# TODO: make it a true template
create_constraint_mode -name coherent-synthesis -sdc_files
↳ {./physical_design/constraints/dlx-synthesized_constraints-strictopt.sdc
}
create_delay_corner -name std-typ -library_set {libsTYP} -rc_corner {standard}
create_delay_corner -name ht-wc -library_set {libsWC} -rc_corner {high}
create_delay_corner -name lt-bc -library_set {libsBC} -rc_corner {low}
create_analysis_view -name default -constraint_mode {coherent-synthesis}
↳ -delay_corner {std-typ}
create_analysis_view -name worst -constraint_mode {coherent-synthesis} -delay_corner
↳ {ht-wc}
create_analysis_view -name best -constraint_mode {coherent-synthesis} -delay_corner
↳ {lt-bc}
set_analysis_view -setup {default} -hold {default}
```

C.3.2 DLX.globals

```
set defHierChar {/}
set delaycal_input_transition_delay {0.1ps}
set fpIsMaxIoHeight 0
set init_gnd_net {gnd}

# Load the structure of the top level entity
# TODO: make it a true template
set init_mmmc_file {./scripts/scripts_template/default.view}

set init_oa_search_lib {}
set init_pwr_net {vdd}
```

```

set init_verilog {./physical_design/designs/dlx-structural-strictopt.v}
set lsgOCPGainMult 1.000000

set LEF_DIR /eda/dk/nangate45/lef
set LEF_list [list ${LEF_DIR}/NangateOpenCellLibrary.lef]

set init_lef_file "${LEF_list}"

set LIB_DIR /eda/dk/nangate45/liberty
set MyTimingLibNom ${LIB_DIR}/NangateOpenCellLibrary_typical_ecsm_nowlm.lib
set MyTimingLibSlow ${LIB_DIR}/NangateOpenCellLibrary_slow_ecsm.lib
set MyTimingLibFast ${LIB_DIR}/NangateOpenCellLibrary_fast_ecsm.lib

set MycapTable ${LEF_DIR}/captables/NCSU_FreePDK_45nm.capTbl

```

C.3.3 Physical Design

```

#=====
# Project      : DLX Processor Implementation
# File         : synthesis_template.tcl
# Description  : TCL file used to run the synthesis of the DLX processor
#
# Group Name   : Group 02
# Members      : Sabina Sarcuni
#               Leonadro Gallina
#               Francesco Mignone
#
# Supervisor   : Mariagrazia Graziano, Giavanna Turvani
# Institution   : Polytechnic of Turin, Italy
#
# Created      : 11 July 2025
# Last Edited  : 11 July 2025
#
# Notes        : - 5-stage pipeline (IF, ID, EX, MEM, WB)
#               - Supports basic instruction set: R-type, I-type, J-type
#               - Hazard detection and forwarding implemented
#               - Designed for simulation in ModelSim/Questasim
#
#=====
# Available variables from the bash script
setMultiCpuUsage -localCpu 8 -cpuAutoAdjust true

# Load the design
source ./scripts/scripts_templates/DLX.globals
# Do the physical design
init_design
getIoFlowFlag
setIoFlowFlag 0
floorPlan -coreMarginsBy die -site FreePDK45_38x28_10R_NP_162NW_340 -r 1.0 0.6 5 5 5
↪ 5

```

```

uiSetTool select
getIoFlowFlag
fit
set sprCreateIeRingOffset 1.0
set sprCreateIeRingThreshold 1.0
set sprCreateIeRingJogDistance 1.0
set sprCreateIeRingLayers {}
set sprCreateIeRingOffset 1.0
set sprCreateIeRingThreshold 1.0
set sprCreateIeRingJogDistance 1.0
set sprCreateIeRingLayers {}
set sprCreateIeStripeWidth 10.0
set sprCreateIeStripeThreshold 1.0
set sprCreateIeStripeWidth 10.0
set sprCreateIeStripeThreshold 1.0
set sprCreateIeRingOffset 1.0
set sprCreateIeRingThreshold 1.0
set sprCreateIeRingJogDistance 1.0
set sprCreateIeRingLayers {}
set sprCreateIeStripeWidth 10.0
set sprCreateIeStripeThreshold 1.0
setAddRingMode -ring_target default -extend_over_row 0 -ignore_rows 0 -avoid_short 0
↳ -skip_crossing_trunks none -stacked_via_top_layer metal10
↳ -stacked_via_bottom_layer metal1 -via_using_exact_crossover_size 1
↳ -orthogonal_only true -skip_via_on_pin { standardcell } -skip_via_on_wire_shape
↳ { noshape }
addRing -nets {vdd gnd} -type core_rings -follow core -layer {top metal9 bottom
↳ metal9 left metal10 right metal10} -width {top 0.8 bottom 0.8 left 0.8 right
↳ 0.8} -spacing {top 0.8 bottom 0.8 left 0.8 right 0.8} -offset {top 1.8 bottom
↳ 1.8 left 1.8 right 1.8} -center 1 -threshold 0 -jog_distance 0
↳ -snap_wire_center_to_grid None
set sprCreateIeRingOffset 1.0
set sprCreateIeRingThreshold 1.0
set sprCreateIeRingJogDistance 1.0
set sprCreateIeRingLayers {}
set sprCreateIeRingOffset 1.0
set sprCreateIeRingThreshold 1.0
set sprCreateIeRingJogDistance 1.0
set sprCreateIeRingLayers {}
set sprCreateIeStripeWidth 10.0
set sprCreateIeStripeThreshold 1.0
set sprCreateIeStripeWidth 10.0
set sprCreateIeStripeThreshold 1.0
set sprCreateIeRingOffset 1.0
set sprCreateIeRingThreshold 1.0
set sprCreateIeRingJogDistance 1.0
set sprCreateIeRingLayers {}
set sprCreateIeStripeWidth 10.0
set sprCreateIeStripeThreshold 1.0

```

```

setAddStripeMode -ignore_block_check false -break_at none -route_over_rows_only
↳ false -rows_without_stripes_only false -extend_to_closest_target none
↳ -stop_at_last_wire_for_area false -partial_set_thru_domain false
↳ -ignore_nondefault_domains false -trim_antenna_back_to_shape none -spacing_type
↳ edge_to_edge -spacing_from_block 0 -stripe_min_length stripe_width
↳ -stacked_via_top_layer metal10 -stacked_via_bottom_layer metal1
↳ -via_using_exact_crossover_size false -split_vias false -orthogonal_only true
↳ -allow_jog { padcore_ring block_ring } -skip_via_on_pin { standardcell }
↳ -skip_via_on_wire_shape { noshape }
addStripe -nets {vdd gnd} -layer metal10 -direction vertical -width 0.8 -spacing 0.8
↳ -set_to_set_distance 20 -start_from left -start_offset 15 -switch_layer_over_obs
↳ false -max_same_layer_jog_length 2 -padcore_ring_top_layer_limit metal10
↳ -padcore_ring_bottom_layer_limit metal1 -block_ring_top_layer_limit metal10
↳ -block_ring_bottom_layer_limit metal1 -use_wire_group 0
↳ -snap_wire_center_to_grid None
setSrouteMode -viaConnectToShape { noshape }
sroute -connect { blockPin padPin padRing corePin floatingStripe } -layerChangeRange
↳ { metal1(1) metal10(10) } -blockPinTarget { nearestTarget } -padPinPortConnect {
↳ allPort oneGeom } -padPinTarget { nearestTarget } -corePinTarget {
↳ firstAfterRowEnd } -floatingStripeTarget { blockring padring ring stripe ringpin
↳ blockpin followpin } -allowJogging 1 -crossoverViaLayerRange { metal1(1)
↳ metal10(10) } -nets { vdd gnd } -allowLayerChange 1 -blockPin useLef
↳ -targetViaLayerRange { metal1(1) metal10(10) }
setPlaceMode -prerouteAsObs {1 2 3 4 5 6 7 8}
setPlaceMode -fp false
place_design
fit
fit
fit
getPinAssignMode -pinEditInBatch -quiet
setPinAssignMode -pinEditInBatch true
editPin -fixOverlap 1 -unit MICRON -spreadDirection clockwise -side Left -layer 1
↳ -spreadType center -spacing 0.14 -pin clk
setPinAssignMode -pinEditInBatch false
getPinAssignMode -pinEditInBatch -quiet
setPinAssignMode -pinEditInBatch true
editPin -fixOverlap 1 -unit MICRON -spreadDirection clockwise -side Left -layer 1
↳ -spreadType center -spacing 0.14 -pin rst
setPinAssignMode -pinEditInBatch false
getPinAssignMode -pinEditInBatch -quiet
setPinAssignMode -pinEditInBatch true
editPin -pinWidth 0.07 -pinDepth 0.07 -fixOverlap 1 -unit MICRON -spreadDirection
↳ clockwise -side Left -layer 1 -spreadType center -spacing 0.14 -pin rst
setPinAssignMode -pinEditInBatch false
setOptMode -fixCap true -fixTran true -fixFanoutLoad false
optDesign -postCTS
optDesign -postCTS -hold
getFillerMode -quiet

```

```

addFiller -cell FILLCELL_X1 FILLCELL_X2 FILLCELL_X4 FILLCELL_X8 FILLCELL_X16
↳ FILLCELL_X32 -prefix FILLER
setNanoRouteMode -quiet -timingEngine {}
setNanoRouteMode -quiet -routeWithSiPostRouteFix 0
setNanoRouteMode -quiet -drouteStartIteration default
setNanoRouteMode -quiet -routeTopRoutingLayer default
setNanoRouteMode -quiet -routeBottomRoutingLayer default
setNanoRouteMode -quiet -drouteEndIteration default
setNanoRouteMode -quiet -routeWithTimingDriven false
setNanoRouteMode -quiet -routeWithSiDriven false
routeDesign -globalDetail
setAnalysisMode -analysisType onChipVariation
setOptMode -fixCap true -fixTran true -fixFanoutLoad false
optDesign -postRoute
optDesign -postRoute -hold
saveDesign DLX

# generate the reports

set_analysis_view -setup {default} -hold {default}
reset_parasitics
extractRC
rcOut -spef DLX.spef -rc_corner standard
redirect -quiet {set honorDomain [getAnalysisMode -honorClockDomains]} > /dev/null
timeDesign -postRoute -pathReports -drvReports -slackReports -numPaths 50 -prefix
↳ DLX_postRoute -outDir ./physical_design/timingReport
redirect -quiet {set honorDomain [getAnalysisMode -honorClockDomains]} > /dev/null
timeDesign -postRoute -hold -pathReports -slackReports -numPaths 50 -prefix
↳ DLX_postRoute -outDir ./physical_design/timingReport
getAnalysisMode -checkType
getAnalysisMode -checkType
get_time_unit
report_timing -machine_readable -max_paths 10000 -max_slack 0.75 -path_exceptions
↳ all -late > top.mtarpt
load_timing_debug_report -name default_report top.mtarpt
verifyConnectivity -type all -error 1000 -warning 50
reportGateCount -level 5 -limit 100 -outfile DLX.gateCount
saveNetlist ./physical_design/DLX.v
all_hold_analysis_views
all_setup_analysis_views
write_sdf -ideal_clock_network ./physical_design/DLX.sdf

exit

```